

# A Sound Interval-MDP Abstraction of an Autonomous Emergency Braking System

Construction, Soundness Certificate, and PRISM Encoding

Heba Al Kayed

School of Informatics, University of Edinburgh

## Abstract

Autonomous systems increasingly place learned perception components inside safety-critical control loops, and certifying such systems demands quantitative guarantees: bounds on the probability of an unsafe event within a given time horizon. Existing verification frameworks for learning-enabled autonomous systems, spanning compositional verification, falsification, and runtime assurance, analyse a system-level model that is taken as given. Constructing that model soundly from the concrete dynamics is itself an obstacle: the state space is continuous, the dynamics are uncertain, and a faithful finite model can be prohibitively large. Our insight is that soundness can be carried per transition and certified on the shipped artifact: we abstract the system as an Interval Markov Decision Process, in which every transition carries a probability interval guaranteed to bracket the transition probabilities of the modelled dynamics, and we certify this containment by an exhaustive test rather than an analytic error bound. We present a modular pipeline that applies the construction to an Autonomous Emergency Braking System under perfect perception, the first track of a programme on verifying learning-enabled cyber-physical systems and synthesising runtime monitors for them, and that emits the abstraction both as modular PRISM components and as a flattened encoding of the closed loop, certified equivalent to them, for robust verification with the Storm model checker [?] at scale. The containment certificate covers a model of roughly two million states and reports zero violations across all cells and actions. Robust model checking certifies a crash probability of exactly one from the imminent-collision scenario and returns saturated brackets for the remaining scenarios at the ten-second horizon, locating the precision limit of the reported resolution. Horizon-resolved verification then locates, per scenario, the largest horizon that resolution decides, and a second certified configuration at finer resolution yields fractional, scenario-discriminating probability brackets for a family of car-following start states.

## Contents

|          |                                                   |          |
|----------|---------------------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                               | <b>2</b> |
| <b>2</b> | <b>Background</b>                                 | <b>4</b> |
| 2.1      | Discrete-time stochastic hybrid systems . . . . . | 4        |
| 2.2      | Interval Markov decision processes . . . . .      | 4        |
| 2.3      | The probabilistic invariance problem . . . . .    | 4        |
| 2.4      | Robust verification of interval models . . . . .  | 5        |
| <b>3</b> | <b>Related Work</b>                               | <b>5</b> |
| <b>4</b> | <b>The Abstraction Procedure</b>                  | <b>6</b> |
| 4.1      | Partitioning the state space . . . . .            | 7        |
| 4.2      | Per-transition probability intervals . . . . .    | 10       |
| 4.3      | Soundness of the abstraction . . . . .            | 21       |

|          |                                                                            |           |
|----------|----------------------------------------------------------------------------|-----------|
| 4.4      | The full pipeline . . . . .                                                | 23        |
| 4.5      | Adaptation of, and departure from, the partition-based framework . . . . . | 24        |
| 4.6      | Choosing the partition resolution . . . . .                                | 24        |
| <b>5</b> | <b>Case Study: An Autonomous Emergency Braking System</b>                  | <b>25</b> |
| 5.1      | System description . . . . .                                               | 26        |
| 5.2      | Implementation architecture . . . . .                                      | 29        |
| 5.3      | Experimental configuration, certificates, and verification . . . . .       | 30        |
| 5.4      | Horizon-resolved verification on the production grid . . . . .             | 34        |
| 5.5      | Resolution sweep . . . . .                                                 | 35        |
| 5.6      | A second certified configuration: the Following window . . . . .           | 35        |
| 5.7      | Future work . . . . .                                                      | 38        |
| <b>6</b> | <b>Conclusion</b>                                                          | <b>39</b> |

## 1 Introduction

Cyber-physical systems that interact with uncertain environments, such as autonomous vehicles, robotic systems, and safety-critical control loops, are naturally modelled as *stochastic hybrid systems* [?], in which real-valued physical quantities evolve under stochastic dynamics and are coupled with discrete control or mode variables. Certifying such systems against safety requirements demands quantitative guarantees of the form

*the probability of an unsafe event within a given time horizon does not exceed a prescribed threshold.*

Probabilistic model checking can answer such questions: given a model of the system, it computes the probabilities of the events of interest and checks them against the required thresholds [?]. Its algorithms require the model to have a finite, or at most countable, state space, and so do not apply directly to a system whose state space is continuous and therefore uncountable. The system must therefore first be *abstracted* into a model the checker can handle, one whose dynamics approximate the original closely enough that verification results obtained on it carry back to the original system. The construction of such abstractions is itself an established research area with a range of approaches [?]. We treat it here as a stepping stone towards the broader goal of formally verifying stochastic hybrid systems of this kind, and it is with that intent that the present work is carried out.

Many of the systems in question are moreover *learning-enabled*: their perception of the environment is produced by trained components such as deep neural networks (DNNs). The research question this programme addresses is: given a concrete, executable model of a learning-enabled control system, how can one automatically construct a finite model that (a) provably preserves quantitative guarantees, (b) composes with a separately quantified perception layer, and (c) supports the offline synthesis of runtime monitors? This paper answers part (a) for the plant and controller under perfect perception, and builds the interfaces on which (b) and (c) rest.

This work is accordingly positioned among the verification frameworks for learning-enabled autonomous systems, surveyed under the Verified AI agenda of [?]. Compositional verification derives system-level guarantees from component-level guarantees and assumptions between the components [?]; falsification searches the input space of a closed-loop system for executions that violate a specification, and has been applied to an AEBS with DNN perception [?]; runtime assurance in the Simplex tradition pairs an untrusted controller with a verified fallback and a monitor that switches between them [?]. [UNVERIFIED: the characterisations of [?], [?], and [?] in this paragraph and the next are to be verified against the sources before circulation.] These frameworks operate on a system-level model that they take as given. The gap this programme

addresses is the construction of that model: from the concrete dynamics of the system we build the finite model automatically, together with a certificate that it soundly represents them.

In other words, the relationship to those frameworks is complementary. Where falsification searches for violating executions and is silent when it finds none, we compute a certified bracket on the violation probability, which covers that case; in compositional terms, we construct the plant-side component that a composition rule takes as given; and in runtime-assurance terms, the bounds computed offline against our abstraction are what license a monitor’s switching decision. [UNVERIFIED: see the note above.]

This paper documents a sound finite-state abstraction for an Autonomous Emergency Braking System (AEBS), a representative stochastic hybrid system with three continuous state variables and a small finite action set. Our abstraction procedure produces an Interval Markov Decision Process (IMDP), a finite model whose transitions carry probability intervals rather than exact values. The intervals let one model hold two kinds of uncertainty at once: the over-approximation introduced when the continuous dynamics are discretised, and the perception uncertainty that later stages of the programme will add. A model checker resolves these intervals by robust, worst-case and best-case, analysis.

Concretely, we present a pipeline that takes an AEBS and, through a sequence of abstraction algorithms, produces an IMDP model of it. The model is emitted as a set of modular PRISM components and is guaranteed to be a sound over-approximation of the underlying AEBS, so that it can be fed directly to a probabilistic model checker, here Storm [?], to check the safety properties of interest. For the AEBS these are bounds on the probability of a crash within a given time horizon, evaluated under standard car-to-car lead-vehicle scenarios [?]. The individual stages of the pipeline are developed in Section 4.

The construction is the first track of the programme, whose end goal is the offline synthesis of deployable runtime monitors: components that sit between the perception components and the controller at run time and guard the system against perception failures, backed by safety bounds computed offline against the abstraction. The programme decomposes into three tracks (Figure 1). In plain terms, Track 1 answers the question: how dangerous is the physics under this controller, if the sensors were perfect? Track 2 answers: how wrong can the camera and radar be, and on which inputs? Track 3 combines the two into: how dangerous is the real system, sensors included?, and it is from this combined model that the monitor is synthesised. Technically, Track 1 is the formal-abstraction track documented by this paper; Track 2 calibrates the learning-enabled perception components into per-input uncertainty tuples; and Track 3 composes the two through a perception-aware IMDP construction step that yields the end-to-end model. This paper is concerned with Track 1 under perfect perception.

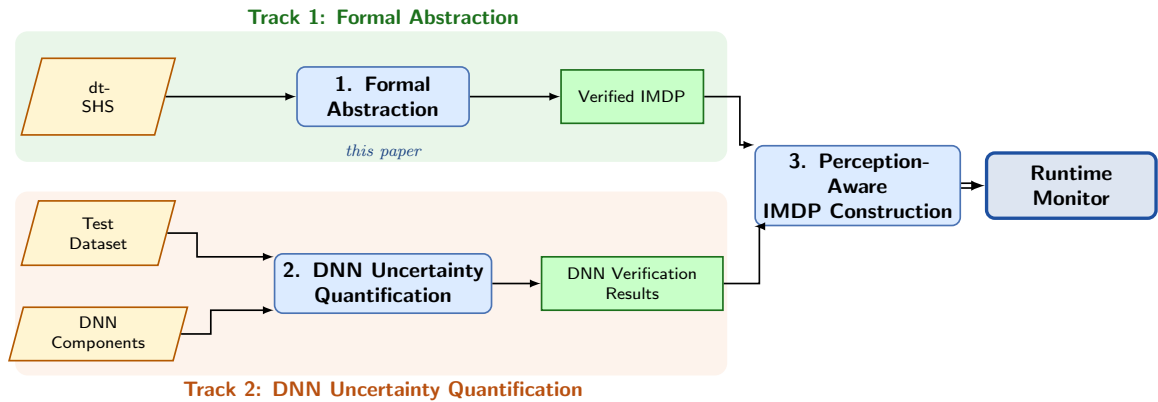


Figure 1: Building blocks of the wider verification programme. This paper documents Track 1.

The remainder of the paper is organised as follows. Section 2 fixes terminology, recalls the probabilistic invariance problem, and states the robust-verification framework on which soundness

rests. Section 3 positions the work relative to the partition-based abstraction literature. Section 4 presents the abstraction procedure and the soundness analysis in detail. Section 5 develops the AEBS case study and reports the certificates and the verification campaigns. Section 6 concludes.

## 2 Background

This section fixes the class of system models we abstract from, the class of finite-state models we abstract to, the verification question we wish to answer, and the framework under which a containing IMDP is a sound surrogate for the continuous system.

### 2.1 Discrete-time stochastic hybrid systems

Following the formalisation of [?, Def. 1.2], a *discrete-time stochastic hybrid system* (dt-SHS) is a tuple

$$\Sigma = (Q, n, X, U, T_x, Y, h), \quad (1)$$

where  $Q = \{q_1, \dots, q_p\}$  is a finite set of discrete modes;  $n : Q \rightarrow \mathbb{N}_{\geq 1}$  assigns to each mode a continuous-state dimension;  $X \subseteq \bigcup_{q \in Q} \{q\} \times \mathbb{R}^{n(q)}$  is the hybrid state space with Borel  $\sigma$ -algebra  $\mathcal{B}(X)$ ;  $U \subseteq \mathbb{R}^m$  is a Borel input space;  $T_x : \mathcal{B}(X) \times X \times U \rightarrow [0, 1]$  is a conditional stochastic kernel;  $Y$  is a Borel output space; and  $h : X \rightarrow Y$  is a measurable output map. The kernel  $T_x$  characterises the one-step transition law: for any  $A \in \mathcal{B}(X)$  and any  $k \in \mathbb{N}$ ,

$$\mathbb{P}(x(k+1) \in A \mid x(k), \nu(k)) = \int_A T_x(dx(k+1) \mid x(k), \nu(k)), \quad (2)$$

where  $x(k) \in X$  is the state at step  $k$  and  $\nu(k) \in U$  is the input applied at that step.

When  $|Q| = 1$  the discrete component is trivial and the model reduces to a discrete-time stochastic control system over a continuous state space [?, Def. 1.3]; this is the form taken by the case study of Section 5.

### 2.2 Interval Markov decision processes

The target of the abstraction is a finite-state model with interval-valued transition probabilities. Following [?], an *Interval Markov Decision Process* (IMDP, also called a *bounded-parameter MDP*) is a tuple

$$\mathcal{I} = (\hat{S}, \hat{U}, \check{P}, \hat{P}), \quad (3)$$

where  $\hat{S}$  is a finite set of states,  $\hat{U}$  a finite set of actions, and  $\check{P}, \hat{P} : \hat{S} \times \hat{U} \times \hat{S} \rightarrow [0, 1]$  are lower- and upper-bound transition functions with  $\check{P}(s, a, s') \leq \hat{P}(s, a, s')$  for every triple. The IMDP represents the set of all MDPs whose exact transition probabilities  $P(s, a, s')$  satisfy  $\check{P}(s, a, s') \leq P(s, a, s') \leq \hat{P}(s, a, s')$  and whose rows sum to one. Verification of an IMDP property is performed against the worst-case (or, dually, best-case) choice of transition distribution consistent with the stored intervals, so the output is itself an interval bracketing the property value over the entire set of consistent MDPs.

### 2.3 The probabilistic invariance problem

For a dt-SHS  $\Sigma$  with state space  $X$ , the *probabilistic invariance* (finite-horizon safety) problem [?] is to compute, for each initial state  $x_0 \in X$ , the probability

$$p_{x_0}(A) = \mathbb{P}(x(k) \in A \text{ for all } k \in [0, N] \mid x(0) = x_0), \quad (4)$$

where  $A \subseteq X$  is the Borel set of safe states,  $N$  a finite horizon, and  $x(k)$  the state under the kernel  $T_x$ . The quantity in (4) can be characterised as the value at time 0 of a backward Bellman recursion over  $A$  [?, Thm. 1]; the recursion involves integrals against  $T_x$  that are generally intractable in closed form, which motivates a finite-state abstraction on which the discrete analogue of the recursion is computable.

## 2.4 Robust verification of interval models

An IMDP denotes a *set* of Markov decision processes, namely all those whose row-stochastic transition probabilities lie within the stored intervals. This is the object studied under the name *Markov set-chain* [?]. Robust verification of a temporal-logic property over an IMDP computes the extremal value of the property over the whole denoted set. The preservation result we rely on is that of [?]: when one model’s transition probabilities are contained, transition by transition, within the intervals of an interval model, the robust PCTL value computed on the interval model brackets the property value of the contained model. Containment of the concrete transition probabilities within the abstract intervals is therefore the single hypothesis under which a verification result on the IMDP transfers to the system being abstracted; the notion of approximate abstraction for stochastic hybrid systems we use originates with [?]. We make this hypothesis precise and discharge it in Section 4.3.

## 3 Related Work

Automated verification and synthesis of stochastic hybrid systems is a broad and active area, surveyed recently by [?]; partition-based finite abstraction is one approach within it, and the one to which our work belongs. That survey also identifies the compositional construction of interval Markov processes as one of its open problems. Two prior constructions are most relevant to ours. The first, [?], introduces approximate abstractions of stochastic hybrid systems: the continuous state space is partitioned, a finite Markov model is built over the cells, and the abstraction is accompanied by an a priori bound on the difference between the abstract and concrete probabilistic-safety values, governed by the regularity of the stochastic kernel and the resolution of the partition. The second, [?], develops adaptive and sequential gridding procedures for the same setting: it refines the partition where the kernel varies most rather than uniformly, and supplies a per-transition range bound (its Equation 3.11), which measures the kernel’s actual variation over a source cell, alongside an additive global error bound that accumulates over the verification horizon and that the refinement is designed to reduce. As that procedure makes explicit, adaptive refinement helps most when the per-cell error is spatially non-uniform, so that resolution can be concentrated where it most reduces the global bound.

The construction documented here is also a partition-based abstraction for a stochastic hybrid system with a finite action set, but it targets an interval-valued model and certifies soundness by per-transition containment rather than by an analytic global bound. From [?] it reuses a single analytic ingredient, the per-transition range bound of Equation 3.11, applied at the integrated-kernel level on the one stochastic dimension; it adopts neither the adaptive gridding nor the additive global bound as a soundness mechanism. The reason is twofold. First, structurally: the AEBS kernel is degenerate, a product of Dirac point masses on the deterministic dimensions and a single Gaussian, so it admits no density on the full state space. The error analysis of [?] rests on Lipschitz continuity of the conditional density, an assumption that line of work states explicitly and that its benchmark systems satisfy by carrying full-support noise on every continuous state variable [?, ?]; on the deterministic dimensions here, the cell-to-cell probability is an indicator that jumps between 0 and 1, which no finite Lipschitz constant describes, so the additive bound is inapplicable there rather than merely loose. Second, quantitatively: on the one dimension where a density does exist, the noise is constant, so the per-row interval width is essentially uniform across the state space, which leaves the adaptive procedure no heterogeneity to exploit and renders the additive bound uninformative at practical horizons; this second obstacle would persist even under full-support constant noise. Degenerate noise of this kind is addressed in a more recent branch of the same literature, which likewise reaches for interval-valued models certified by per-transition bounds: [?] abstracts systems  $x^+ = f(x, w)$  whose noise domain has dimension decoupled from the state’s, by partitioning the noise domain and bounding each transition through set-valued images, and [?, ?] combine mixed-monotone reachable-set computation on the deterministic dynamics with interval-valued transition bounds. Our construction belongs

to this interval branch while retaining the per-transition range bound of [?] on the dimension where its density premise holds. Soundness is obtained instead from per-transition containment under the Markov set-chain preservation result [?, ?], in the approximate-abstraction sense of [?], with the additive bound retained only as a diagnostic. The handling of the deterministic dimensions and the full rationale are developed where they arise (Sections 4.2 and 4.5). The choice of an interval-valued target model is motivated, as in Section 1, by the need to carry both partition-induced and perception-induced uncertainty through to the verifier.

## 4 The Abstraction Procedure

This section presents the construction. Section 4.1 partitions the continuous state space into the states of the IMDP; Section 4.2 computes the probability intervals attached to its transitions; Section 4.3 establishes soundness through the containment certificate, the exhaustive per-transition test by which the IMDP is proved to over-approximate the continuous system; Section 4.4 composes the steps into the end-to-end pipeline that emits the abstraction for the verifier.

The procedure applies to the following class of systems, which mixes deterministic and stochastic state variables; the two kinds of dimension are treated by different mechanisms at every stage of the construction.

**Assumption 1** (System class). *The system is a dt-SHS (1) with  $|Q| = 1$ , so that the hybrid state reduces to the continuous state  $s \in \mathcal{S}$ , with  $\mathcal{S} \subseteq \mathbb{R}^n$  bounded and the action set  $U$  finite. For every action  $a \in U$ :*

(i) Additive independent noise. *One step from state  $s$  under action  $a$  lands at*

$$s' = \mu(s, a) + w, \quad (5)$$

where

- $s' \in \mathbb{R}^n$  is the successor state;
- $\mu : \mathcal{S} \times U \rightarrow \mathbb{R}^n$  is the one-step mean map, giving the successor before noise;
- $w \in \mathbb{R}^n$  is the noise vector; its components  $w_d$  are independent of one another. In each deterministic dimension,  $w_d = 0$ , written  $\sigma_d = 0$ . In each stochastic dimension,  $w_d$  follows the normal distribution with mean zero and variance  $\sigma_d^2$ , written  $w_d \sim \mathcal{N}(0, \sigma_d^2)$ , where the standard deviation  $\sigma_d > 0$  is constant over  $\mathcal{S}$ .

Equivalently, the one-step kernel factorises across the dimensions,

$$T_x(A_1 \times \cdots \times A_n \mid s, a) = \prod_{d=1}^n T_{x,d}(A_d \mid s, a), \quad (6)$$

where

- $T_x$  is the one-step kernel of (2);
- $A_d \subseteq \mathbb{R}$  is an interval of values for coordinate  $d$  of the successor state;
- $A_1 \times \cdots \times A_n$  is the region of states that meet all  $n$  of these interval conditions at once, one per coordinate;
- $T_{x,d}(A_d \mid s, a)$  is the probability that coordinate  $d$  of the successor state falls in  $A_d$ . In a deterministic dimension the coordinate is exactly  $\mu_d(s, a)$ , so this probability is 1 if  $\mu_d(s, a)$  lies in  $A_d$  and 0 if it does not. In a stochastic dimension the coordinate is  $\mu_d(s, a)$  plus the noise  $w_d$ , so it is the probability that this noisy value lands in  $A_d$ .

(ii) Continuity. *The map  $s \mapsto \mu(s, a)$  is continuous on  $\mathcal{S}$ .*

(iii) Component-wise monotonicity. Take any one coordinate of the state and change it upward, keeping the rest of the state as it is. Each coordinate of  $\mu(s, a)$  then responds in a fixed way: it only ever rises (or stays put), or it only ever falls (or stays put). Which of the two happens depends on the pair of coordinates involved, but never on where in  $\mathcal{S}$  the change is made.

The additive form (5) is the standard shape in this abstraction literature [?]; the component-wise monotonicity of part (iii) is the property exploited for corner-based reachable-set computation in [?]; the mixed Dirac and Gaussian split is the restriction this construction is designed for. That the case study satisfies the assumption is established in Section 5; the constancy of  $\sigma_d$  in part (i) is additionally checked at construction time (Algorithm 2).

#### 4.1 Partitioning the state space

Following the partition-based abstraction approach of [?, ?], the continuous state space is covered by a finite grid of non-overlapping cells. Terminology is fixed as follows; Figure 2 depicts the three notions. The space  $\mathcal{S}$  belongs entirely to the concrete system, and a *continuous state* is a point of  $\mathcal{S}$ . A *cell* is a region of  $\mathcal{S}$  delimited by the grid. A *state* is an element of the finite model and carries no geometry of its own. The two sides are connected by mapping each cell to one state. The finite model occupying state  $i$  means that the concrete system lies at some point of the cell  $A_i$ , with no record of which; the transition intervals of Section 4.2 are constructed to be valid for every such point.

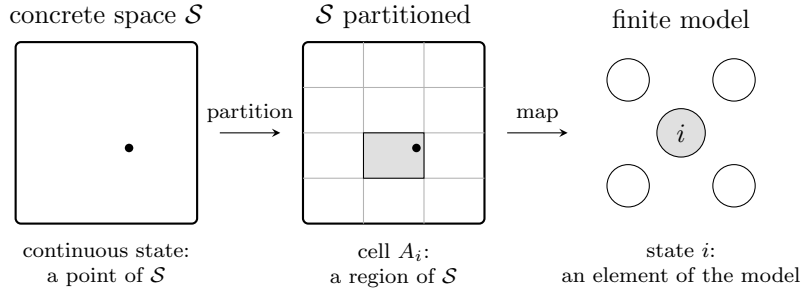


Figure 2: Continuous states, cells, and states. Left: a continuous state is a point of the concrete space  $\mathcal{S}$ . Centre: the partition delimits cells; the point has not moved and lies in cell  $A_i$ . Right: a state is an element of the finite model; cell  $A_i$  is mapped to state  $i$ .

The complete state set of the model, including its absorbing states, is assembled at the end of this subsection. The bounded state space is covered as

$$\mathcal{S} \subseteq \bigcup_{i=1}^M A_i, \quad A_i \cap A_j = \emptyset \text{ for } i \neq j, \quad (7)$$

where

- $\mathcal{S} \subseteq \mathbb{R}^n$  is the continuous state space;
- $A_i$  is the  $i$ -th cell, a bounded region of  $\mathcal{S}$  containing uncountably many continuous states;
- $M$  is the total number of cells.

The coordinates of  $\mathcal{S}$  are indexed  $d = 1, \dots, n$  and called *dimensions*. Each cell is specified by one interval of values per dimension and contains exactly the continuous states whose coordinate in each dimension lies in that dimension's interval; a region of this form is called an *axis-aligned*

box, and Figure 3 shows one cell with its defining intervals.

Each interval is *half-open*, containing its lower endpoint and excluding its upper endpoint,

$$A_i = \prod_{d=1}^n [\text{lo}_{i,d}, \text{hi}_{i,d}), \quad (8)$$

where

- $\text{lo}_{i,d}$  and  $\text{hi}_{i,d}$  are the lower and upper endpoints of cell  $i$ 's interval in dimension  $d$ .

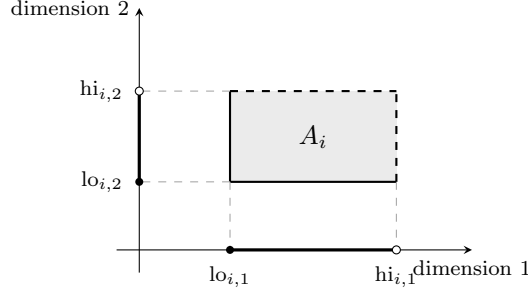


Figure 3: One cell and its defining intervals, drawn for  $n = 2$ . The cell  $A_i$  contains exactly the continuous states whose coordinate in each dimension lies in that dimension's interval. Solid edges belong to the cell and dashed edges do not: each interval contains its lower endpoint (filled) and excludes its upper endpoint (open).

A point lying on a boundary shared by two cells therefore belongs to exactly one of them, the cell whose lower endpoint it is. This is what makes the cells of (7) pairwise disjoint while covering  $\mathcal{S}$ : every continuous state lies in exactly one cell.

The grid is uniform: in each dimension  $d$ , all cells have the same width  $r_d$ , the grid *resolution* in that dimension. The cells of a dimension are delimited by a sorted list of boundary values, its *edges*: the first cell lies between the first and second edge, the second cell between the second and third, and so on, as Figure 4 shows. The edges of dimension  $d$  are

$$\text{edges}_d = \left( \ell_d - \frac{r_d}{2}, \ell_d + \frac{r_d}{2}, \ell_d + \frac{3r_d}{2}, \dots, h_d + \frac{r_d}{2} \right), \quad (9)$$

where

- $r_d$  is the resolution in dimension  $d$ ;
- $\ell_d$  and  $h_d$  are the lower and upper bounds of dimension  $d$ ;
- $\text{edges}_d$  is the resulting list: it starts half a cell below  $\ell_d$  and advances in steps of  $r_d$  until half a cell above  $h_d$ .

Placing the values  $\ell_d, \ell_d + r_d, \dots, h_d$  in cell interiors rather than on cell boundaries has two effects: floating-point rounding of a value computed at such a continuous state cannot move it across a cell boundary, and the centre of a cell can coincide exactly with the continuous state it represents. Because the outermost cells extend half a cell beyond the bounds, the union of the cells strictly contains  $\mathcal{S}$ ; (7) therefore states a covering rather than an equality.

Writing  $n_d$  for the number of cells along dimension  $d$ , the grid total cells are described as follows

$$M = \prod_{d=1}^n n_d \quad (10)$$

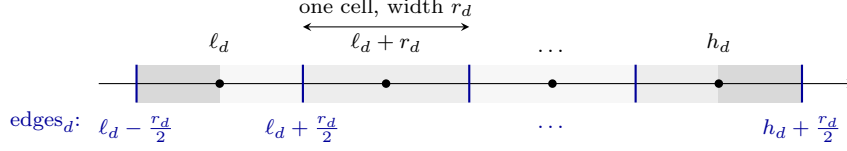


Figure 4: The edges of one dimension, drawn for four cells. The blue ticks are the entries of  $\text{edges}_d$  in (9); each pair of consecutive edges delimits one cell, shown as alternating strips of width  $r_d$ . The black points are the cell centres. The first edge sits half a cell below the lower bound  $\ell_d$ , so the bounds  $\ell_d$  and  $h_d$  fall on the centres of the outermost cells; the darker strips are the halves of those cells extending beyond the bounds.

Each cell is assigned one integer state identifier. The cells are enumerated in *row-major order*, the convention in which the index of the last dimension varies fastest, and the per-dimension cell indices  $(k_1, \dots, k_n)$  are flattened into a single integer. Distinct index tuples receive distinct identifiers, and each identifier corresponds to exactly one cell. The centre  $c_i$  of each cell is recorded alongside it. The centre plays no role in the transition construction of Section 4.2, which bounds the dynamics over every continuous state of a cell rather than at any single representative point; it is used where the pipeline quantises the controller and the state labelling (Section 4.4).

The construction also records a *partition fingerprint*, a hash of the exact edge values of every dimension; two runs operate on the same partition exactly when their fingerprints match.

The model contains *absorbing* states of two kinds. A state is absorbing when, under every action, its only transition is the certain self-loop  $[1, 1]$  to itself: a run that enters an absorbing state remains there. The first kind handles a designated *terminal region* of  $\mathcal{S}$ : a region in which the property under study is already decided, so the dynamics need not be tracked further. One grid cell inside the region is designated its representative and made absorbing, and the interval construction of Section 4.2 routes to the representative every transition whose reachable set meets the region; a terminal region therefore introduces no state beyond the  $M$  grid cells. The construction designates a single terminal region, as the case study of Section 5 has one outcome decided at entry; further such regions would be handled identically, each through its own absorbing representative and label. The second kind is the *safe sink*, one additional state appended after the  $M$  grid cells. The dynamics are not confined to the modelled window  $\mathcal{S}$ , and the sink receives the probability mass of transitions that leave the grid. The kinds carry distinct labels for the verifier: mass absorbed at a terminal representative bears on the property being checked, whereas mass in the sink records departure from the modelled window, and separate labels keep the contributions separately quantifiable.

Algorithm 1 records this partitioning step: it lays down the cell edges of (9) in each dimension, enumerates the Cartesian product of cell indices in row-major order, designates the terminal-region representative, and appends the safe sink.

---

**Algorithm 1** Grid construction.

---

**Require:** bounds  $(\ell_d, h_d)$  and resolution  $r_d$  per dimension; terminal region

**Ensure:** partition  $\{A_i\}_{i=1}^M$ , identifiers, centres, fingerprint, designated absorbing states

- 1: per dimension, compute the cell count and set  $\text{edges}_d$  (9)
  - 2: enumerate cells; assign row-major identifiers
  - 3: record cell centres and the partition fingerprint
  - 4: designate the terminal representative; reserve the sink identifier
  - 5: **return** partition, identifiers, centres, fingerprint, absorbing states
-

## 4.2 Per-transition probability intervals

Each cell  $A_i$  of Section 4.1 corresponds to one state, and both are indexed by  $i$ . Transitions of the IMDP connect states, but the one-step law of the concrete system acts on the continuous states inside the cells: for a fixed action  $a \in U$  and target cell  $A_j$ , the one-step probability

$$T_x(A_j \mid s, a), \quad s \in A_i, \quad (11)$$

is a function of the source state  $s$  and generally takes different values across the cell, while the IMDP carries a single transition from state  $i$  to state  $j$ . The construction therefore stores, for each non-absorbing source cell  $A_i$ , each action  $a \in U$ , and each reachable target cell  $A_j$ , an interval  $[p_{\min}, p_{\max}]$  satisfying

$$p_{\min} \leq T_x(A_j \mid s, a) \leq p_{\max} \quad \text{for every } s \in A_i. \quad (12)$$

This uniform containment over the source cell is the hypothesis under which verification results transfer from the IMDP to the continuous system [?] (Section 4.3). The interval for a transition is built in three steps. First, bound the region that the dynamics can send the source cell to; this bound is the *image box*. Second, for each dimension, bracket the probability that the successor's coordinate lands in the target cell's interval; the deterministic and stochastic dimensions of Assumption (i) are bracketed by separate mechanisms. Third, multiply the per-dimension brackets, which by the factorisation (6) brackets the transition probability itself.

**The image box.** By Assumption (i), a step from  $s$  under  $a$  lands at the mean point  $\mu(s, a)$  plus the per-dimension noise  $w$ . Applying the mean map to every state of the source cell gives the *image set*

$$\mu(A_i, a) = \{ \mu(s, a) : s \in A_i \}, \quad (13)$$

the set of mean points to which the cell's states are sent (Figure 6). The image set is in general not aligned to the grid and consists of uncountably many points, so it is not enumerated directly. The construction encloses it in the smallest axis-aligned box that contains it, the *image box*

$$B_i^a = \prod_{d=1}^n [\mu_d^{\text{lo}}, \mu_d^{\text{hi}}], \quad (14)$$

where

- $B_i^a$  is the image box of source cell  $A_i$  under action  $a$ ;
- $\mu_d^{\text{lo}}$  and  $\mu_d^{\text{hi}}$  are the smallest and largest values that coordinate  $d$  of the mean  $\mu(s, a)$  takes as  $s$  ranges over the cell  $A_i$ .

The box contains the image set, so any transition bound computed against the box holds for every continuous state of the cell; and it is a product of one-dimensional intervals, which is the form the per-dimension factorisation of (6) requires.

Computing  $\mu_d^{\text{lo}}$  and  $\mu_d^{\text{hi}}$  directly would mean searching over the uncountably many continuous states of the cell. Component-wise monotonicity (Assumption (iii)) removes that search. In one dimension the principle is elementary: a monotone function on an interval attains its smallest and largest values at the interval's two ends. Extending this to a cell requires the cell's counterpart of the interval ends. The cell  $A_i$  has one interval per dimension (8); the interval in dimension  $d$  runs from  $\text{lo}_{i,d}$  to  $\text{hi}_{i,d}$ , and these two values are its *endpoints*. A *corner* of the cell is a point whose coordinate in every dimension is one of that dimension's two endpoints. Figure 5 shows the four corners of a two-dimensional cell, each labelled by the pair of endpoints that builds it; with two possible endpoints in each of  $n$  dimensions, a cell has  $2^n$  corners.

Consider one coordinate of the next-state mean  $\mu(s, a)$ . By Assumption (iii), raising any one coordinate of the current state  $s$  moves it in one fixed direction. To make the chosen coordinate

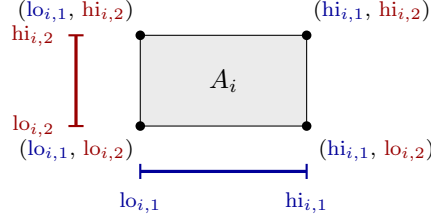


Figure 5: The corners of a cell, for  $n = 2$ . The cell's interval in dimension 1 (blue, below) has the two endpoints  $lo_{i,1}$  and  $hi_{i,1}$ ; its interval in dimension 2 (red, left) has the endpoints  $lo_{i,2}$  and  $hi_{i,2}$ . Each corner pairs one blue endpoint with one red endpoint, as its label shows; two choices per dimension give  $2^n = 4$  corners.

of  $\mu$  as large as possible over the cell, set each coordinate of  $s$  to whichever of its two endpoints moves it upward; the point built from those choices is, by construction, a corner. The smallest value arises in the same way with the opposite choices. Both extreme values are therefore found among the corner evaluations, so scanning all  $2^n$  corners finds them without needing to know which direction any coordinate moves:

$$[\mu_d^{\text{lo}}, \mu_d^{\text{hi}}] = \left[ \min_{c \in \text{corners}(A_i)} \mu_d(c, a), \max_{c \in \text{corners}(A_i)} \mu_d(c, a) \right], \quad d = 1, \dots, n, \quad (15)$$

where

- $\text{corners}(A_i)$  is the set of the  $2^n$  corners of cell  $A_i$ ;
- $\mu_d(c, a)$  is coordinate  $d$  of the next-state mean, the mean map evaluated at corner  $c$  under action  $a$ .

Bounding the image of a box through corner evaluations is the standard construction for monotone systems; the mixed-monotone literature refines it to two evaluations of an auxiliary function [?]. Every transition interval constructed below is computed against this box.

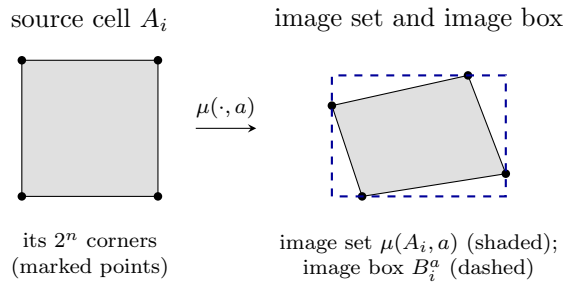


Figure 6: The image box. Left: the source cell, drawn as in Figure 2, with its  $2^n$  corners marked as in Figure 5. Right: the mean map sends the cell to its image set (13); the marked points are the images of the corners. By Assumption (iii) the smallest and largest value of each coordinate over the image is attained at a corner image, so the dashed box they span, the image box (14), encloses the whole image set.

The image box is an intermediate object: it determines the transitions of one source cell under one action and is then discarded, and it is never itself a state of the IMDP. It typically intersects several of the fixed target cells of Section 4.1 (Figure 7), and each target cell the box intersects becomes one transition of the source cell, carrying its own probability interval computed against the box; the cells themselves remain disjoint, it is the box that spans several of them. Because the box is an interval in every dimension, the intersected grid cells form a

contiguous, axis-aligned block in index space: a source cell transitions into a block of target cells, possibly together with the terminal representative and the safe sink, and into no grid target outside the block.

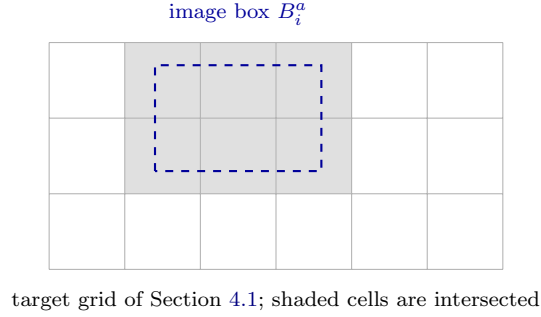


Figure 7: The image box laid over the target grid. The grid is the fixed partition of Figure 2; the dashed box is the image box of Figure 6, in general not aligned to the grid. Each of the six shaded cells it intersects becomes one transition of the source cell; the box is discarded once the transitions are recorded.

A single box encloses the image because the image set is connected: by [Assumption \(ii\)](#) the mean map is continuous for each fixed action, and a continuous map sends the connected source cell to a connected image set. Continuity is required of the dynamics under each fixed action only. The action-selection logic of a controller is typically discontinuous across the state space, and that discontinuity never enters an image-box computation: it is confined to the separate controller component, which only decides which action's transitions apply to each cell ([Section 4.4](#)). Generically, if the concrete system has hybrid dynamics, then a hybrid jump could split the image into disjoint pieces, and the construction would then need a union of image boxes per cell; computing per action is what removes that case here.

**Per-dimension factor intervals.** The factorisation (6) of [Assumption \(i\)](#), instantiated at the target cell, reads

$$T_x(A_j \mid s, a) = \prod_{d=1}^n T_{x,d}([\ell_d^j, h_d^j] \mid s, a), \quad (16)$$

where

- $[\ell_d^j, h_d^j]$  is the target cell's interval in dimension  $d$ , as in (8), so that  $A_j = \prod_d [\ell_d^j, h_d^j]$ ;
- $T_{x,d}$  is the one-dimensional factor of (6).

Each factor on the right of (16) is bracketed by an interval valid for every  $s \in A_i$ , and the transition interval is the product of the factor brackets, Equation (26). The two kinds of dimension give two kinds of factor bracket.

In a *deterministic* dimension ( $\sigma_d = 0$ ) the factor is the indicator of the event  $\mu_d(s, a) \in [\ell_d^j, h_d^j]$ : it equals 1 for a continuous state whose mean lands inside the target interval and 0 otherwise. Over the source cell, the mean  $\mu_d(s, a)$  ranges exactly over the image interval  $[\mu_d^{\text{lo}}, \mu_d^{\text{hi}}]$ , so the tightest bracket valid for every continuous state of the cell is

$$[f_{\text{lo}}, 1], \quad f_{\text{lo}} = \begin{cases} 1 & \text{if } \mu_d^{\text{lo}} \geq \ell_d^j \text{ and } \mu_d^{\text{hi}} < h_d^j, \\ 0 & \text{otherwise,} \end{cases} \quad (17)$$

where

- $f_{\text{lo}}$  is the lower bound of the factor bracket; the upper bound is 1 for every enumerated target;

- the first case holds when the image interval lies wholly inside the half-open target interval, the second when it touches or crosses one of its boundaries.

A factor of  $[1, 1]$  records a target *certainly* reached in that dimension: the image lies at or above the lower boundary and strictly below the upper one, so the mean of every continuous state lands in the half-open target interval. A factor of  $[0, 1]$  records a target only *possibly* reached, and arises in two geometries. If the image *crosses* a boundary of the target interval, some continuous states of the source cell reach the target and others do not, and  $[0, 1]$  is the exact range of the indicator over the cell. If the image only *touches* the upper boundary  $h_d^j$ , the factor  $[0, 1]$  is sound but may be conservative, for the reason given next. Only cells the image interval crosses or touches are enumerated, and each receives the upper bound 1. The upper inequality in (17) is strict because the cells are half-open (Section 4.1): a mean landing exactly on  $h_d^j$  belongs to the cell above, not to the target. Touching images arise routinely, not only through rounding: whenever a dimension moves by a fixed amount  $\delta_d$  equal to the cell width  $r_d$ , a source cell aligned with the grid maps exactly onto its neighbouring cell, and the upper endpoint of the image lands precisely on the shared boundary (Figure 8). A non-strict test would record such a touching image as certain, with lower bound 1; yet a continuous state whose mean sits exactly on  $h_d^j$  reaches the target with probability 0, so the recorded lower bound would exceed the true probability and violate (12). The strict test records  $[0, 1]$  instead, which is sound in every case and conservative only when no state of the half-open source cell attains the boundary.

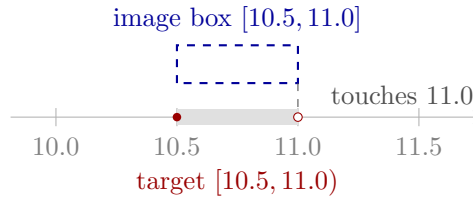
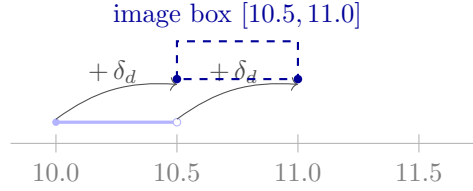
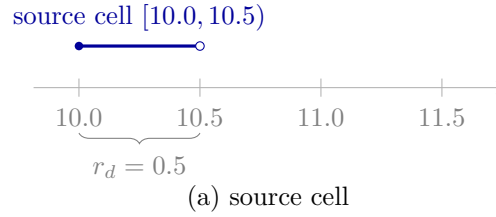


Figure 8: A touching image from an aligned step. The step  $\delta_d = r_d = 0.5$  sends both corners of the source cell onto grid lines, so the image box touches the excluded upper boundary of the target and the factor is  $[0, 1]$ .

In a *stochastic* dimension ( $\sigma_d > 0$ ) each step splits into a prediction and a landing. The mean map sends the continuous state to a single predicted next value, exactly as in a deterministic

dimension. The landing is then a random draw from a Gaussian density centred on the prediction, with standard deviation  $\sigma_d$  (Figure 9). The prediction is a function of the state alone; the spread around it is the noise. Three terms are kept distinct throughout: the *prediction*, where a density is centred; the *target window*, the half-open interval  $[\ell_d^j, h_d^j)$  of a candidate cell; and the *cell middle*, the midpoint of that window. The factor for a target is the landing chance,

$$g(\mu) = \Phi\left(\frac{h_d^j - \mu}{\sigma_d}\right) - \Phi\left(\frac{\ell_d^j - \mu}{\sigma_d}\right), \quad (18)$$

where

- $g(\mu)$  is the chance that the landing falls in the target window  $[\ell_d^j, h_d^j)$  when the prediction is  $\mu$ ;
- $\mu$  is the prediction, the value the mean map assigns to the continuous state;
- $\sigma_d$  is the noise level of the dimension, the standard deviation of the Gaussian, constant over the state space;
- $\Phi$  is the standard normal cumulative distribution function:  $\Phi(x)$  is the chance that a standard Gaussian draw falls below  $x$ , a tabulated function available in every numerical library.

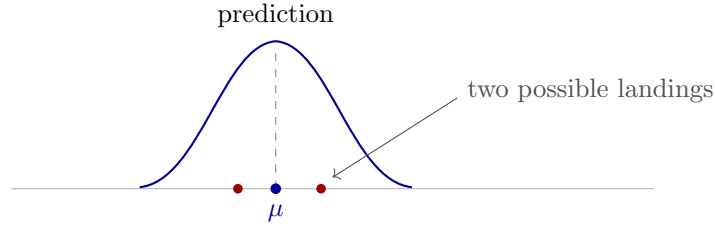


Figure 9: Prediction and landing. The mean map fixes the prediction  $\mu$ ; the density is fixed to it; a landing is a draw from the density. The construction never draws landings: it evaluates their chances in closed form through (18).

A source cell contains a continuum of continuous states, and each state has its own prediction, so the cell owns a family of identical densities whose positions differ. The corner evaluation (15), the same one used for the deterministic dimensions, returns the smallest and largest prediction, and by the monotone continuity of the mean map every value between them is the prediction of some state in the cell. The attainable predictions therefore fill the image interval  $[\mu_d^{\text{lo}}, \mu_d^{\text{hi}}]$  exactly, and no prediction outside it is attainable (Figure 10).

The abstraction must attach one factor to the whole cell, valid for every state in it. Different states have different predictions, hence different landing chances, so the factor is the range of the landing chance over the attainable predictions: from the smallest chance attained by any state of the cell to the largest. Both extremes are located in closed form, because the landing chance depends on the prediction in a simple way: it is largest when the prediction equals the cell middle,

$$\mu^\star = \frac{\ell_d^j + h_d^j}{2}, \quad (19)$$

and it decreases as the prediction moves away from the cell middle in either direction, since the density is symmetric and places less mass on the window the further its centre lies from it. The extremes over the image interval follow immediately. The smallest chance is at the end of the interval farther from the cell middle,

$$m = \min\{g(\mu_d^{\text{lo}}), g(\mu_d^{\text{hi}})\}, \quad (20)$$

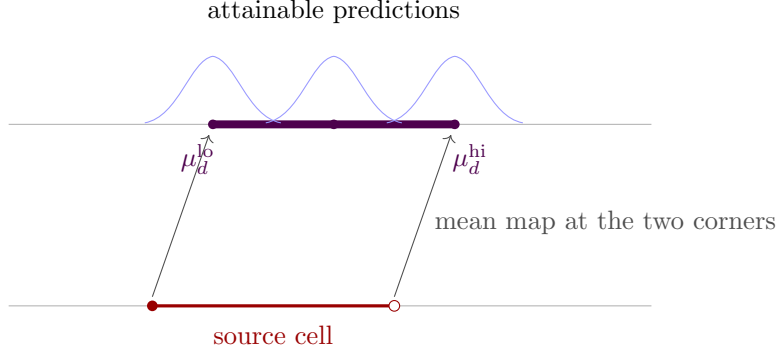


Figure 10: The attainable predictions. The mean map at the two corners of the source cell bounds the predictions; every value between is attained by some state of the cell, and none outside is. Each prediction on the interval carries its own density; three are drawn.

and the largest is at the attainable prediction closest to the cell middle,

$$M = g(\bar{\mu}), \quad (21)$$

where

- $\bar{\mu}$  is the attainable prediction closest to the cell middle:  $\mu^*$  itself if the image interval contains it, the nearer end of the interval otherwise.

The range  $[m, M]$  is the per-transition bound of [?, Eq. 3.11]. Both cases of  $\bar{\mu}$  arise for a single source cell, because neighbouring targets have different middles. Figure 11 places two candidate targets against one image interval: the middle of the first lies inside the interval, the middle of the second outside. Figure 12 then computes the bracket for each target in turn.

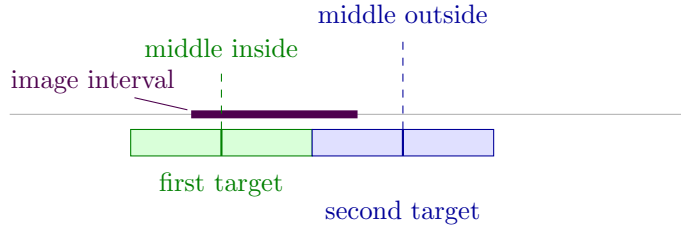


Figure 11: Two candidate targets against one image interval: the first target (green) and the second (blue), each with its middle marked. The first target’s middle lies inside the interval; the second’s lies outside it. The two targets are carried through Figure 12.

Each candidate target is bracketed independently, over the same image interval; repeating the computation per target gives one bracket per transition. The brackets are bounds, not shares of a fixed total: a target whose window lies away from the image interval receives a bracket small at both ends, with the mass appearing in the transitions to the cells the image covers.

The candidate targets are the cells beneath the image interval extended on each side by the noise reach  $n_{\text{cut}} \sigma_d$ , where  $n_{\text{cut}}$  is a cutoff parameter. Cells beyond the window are assigned probability zero, an approximation of at most  $\Phi(-n_{\text{cut}})$  beyond each end of the window, per transition;  $n_{\text{cut}}$  is chosen large enough that this mass lies below the numerical tolerance of the containment certificate and the precision of the model checker. The experiments use  $n_{\text{cut}} = 6$ , for which the ignored mass is of order  $10^{-9}$  (Section 5.3).

**State-independence of the noise.** The bracket (20)–(21) evaluates  $g$  at a single noise magnitude  $\sigma_d$  per source cell. It is therefore valid only under the state-independence of the noise

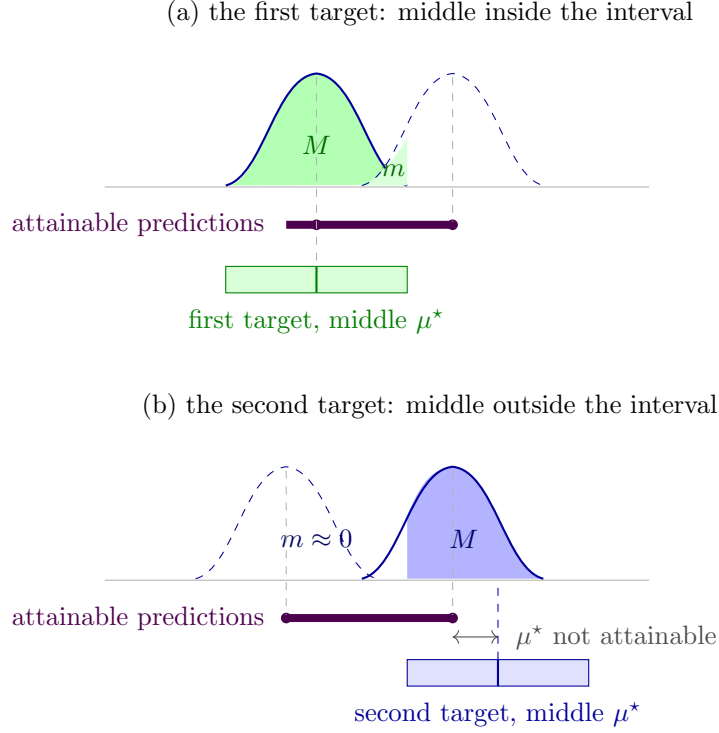


Figure 12: The bracket computed for each target of Figure 11, over the same image interval. Each density stands on its placement, marked by a violet dot on the interval of attainable predictions and tied to the peak by a dashed guide; the shaded area over the target, in the target’s colour, is the landing chance. (a) The first target’s middle is attainable, so the solid density is placed there,  $\bar{\mu} = \mu^*$ , giving the largest chance  $M$  (21); the dashed density at the far end gives the smallest,  $m$  (20). (b) The second target’s middle is not attainable: the solid density is placed at the interval’s end, the closest allowed position, and the far-end density does not reach the target, so  $m$  is negligible.

required by [Assumption \(i\)](#): were  $\sigma_d$  to vary with the state,  $g$  would have to be ranged over the spread of  $\sigma_d$  across the cell as well as over the image. Algorithm 2 checks this part of the assumption at construction time: it records the noise at every corner of the source cell, requires the values to agree to numerical tolerance, and aborts otherwise.

**Truncation of the stochastic tail.** The grid covers a bounded interval  $[b_{\min}, b_{\max}]$  of the stochastic dimension, its *grid band*. The cells of the dimension partition the band:  $b_{\min}$  is the lower edge of the first cell and  $b_{\max}$  the upper edge of the last. A density whose prediction lies near an end of the band places part of its mass past that end, outside the grid (Figure 13). The abstraction assumes that the dimension remains inside the band, so this mass must be removed from the factors. Each factor is divided by the probability that the landing lies inside the band. Away from the ends of the band this probability is 1 and the factor is unchanged; near an end it is smaller than 1, and the division raises the factor to compensate for the removed mass.

The divisor is the probability that the landing lies inside the band,

$$\Pi(\mu) = \Phi\left(\frac{b_{\max} - \mu}{\sigma_d}\right) - \Phi\left(\frac{b_{\min} - \mu}{\sigma_d}\right), \quad (22)$$

where

- $\Pi(\mu)$  is the probability that a landing with prediction  $\mu$  lies inside the band  $[b_{\min}, b_{\max}]$ .

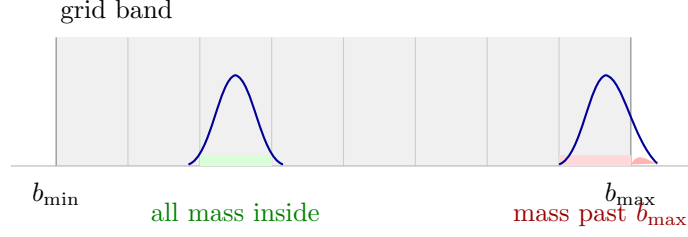


Figure 13: Truncation of the stochastic tail. The cells of the stochastic dimension partition the grid band  $[b_{\min}, b_{\max}]$ . For an interior cell all mass lands inside the band and the division changes nothing; for a cell at an end of the band, part of the mass falls past the edge and the division compensates for its removal.

For a single continuous state, the stored probability of a target cell is the conditional probability of reaching it given that the landing lies inside the band. This conditional probability is the quotient of the landing chance by the divisor,

$$\frac{g(\mu)}{\Pi(\mu)}, \quad (23)$$

where

- $g(\mu)$  and  $\Pi(\mu)$  are evaluated at the prediction  $\mu$  of that state.

Every value in the image interval is the prediction of some state of the source cell, and each such value has its own divisor. The stored factor must therefore bracket the quotient over the whole image interval. The divisor is largest for a prediction at the midpoint of the band and shrinks toward the band's ends. Over the image interval, the largest divisor is therefore found at the point of the interval closest to the band midpoint, and the smallest at the end of the interval farthest from it. The lower end of the stored factor is divided by the largest divisor and the upper end by the smallest, so that the lower end can only decrease and the upper end only increase:

$$\left[ m/\Pi_{\text{hi}}, M/\Pi_{\text{lo}} \right], \quad (24)$$

where

- $\Pi_{\text{lo}}$  and  $\Pi_{\text{hi}}$  are the smallest and largest values of the divisor over the image interval.

The stored ends are safe for every state of the cell. Each state's conditional probability is built from a landing chance no smaller than  $m$  and no larger than  $M$ , and a divisor no smaller than  $\Pi_{\text{lo}}$  and no larger than  $\Pi_{\text{hi}}$ , so it cannot fall below the stored lower end nor exceed the stored upper end:

$$\frac{m}{\Pi_{\text{hi}}} \leq \frac{g(\mu)}{\Pi(\mu)} \leq \frac{M}{\Pi_{\text{lo}}} \quad \forall \mu \in [\mu_d^{\text{lo}}, \mu_d^{\text{hi}}], \quad (25)$$

where

- $g(\mu)$  and  $\Pi(\mu)$  are the landing chance (18) and the divisor (22) at the prediction  $\mu$ ;
- $m$  and  $M$  are the ends of the landing-chance bracket (20)–(21), and  $\Pi_{\text{lo}}$  and  $\Pi_{\text{hi}}$  the smallest and largest divisor over the image interval, as in (24);
- $[\mu_d^{\text{lo}}, \mu_d^{\text{hi}}]$  is the image interval (15).

The two factor types are visible in the generated model. A deterministic dimension has no randomness, so its factor places the whole outcome at one point. The ends of that factor are

therefore 0 or 1, and its value can turn on which cell owns a shared boundary. A stochastic dimension spreads its outcome as a density, and a single point carries no mass under a density, so boundary ownership does not affect its factor, and the factor's ends are fractional. A transition's interval is the product (26) of one factor per dimension, and every fractional entry in the model is the stochastic factor showing through that product (Figure 14).

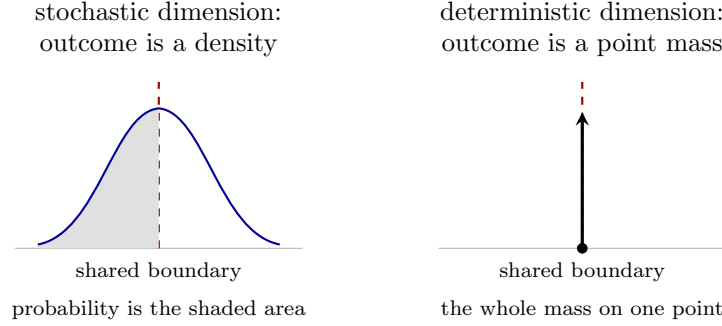


Figure 14: The two factor types. In a stochastic dimension (left) the outcome spreads as a density; a single point carries no mass, so which cell owns a shared boundary does not affect the factor. In a deterministic dimension (right) the outcome is a point mass, drawn on the boundary; which cell owns the boundary decides which cell receives the mass, under the half-open convention of Section 4.1.

**Terminal routing.** A designated terminal region absorbs the runs that enter it. The routing fills the row of any source cell whose image meets the region, given a membership test that classifies the image as wholly inside, partly inside, or outside the region. When the image is wholly inside, no factors are computed: the row is a single transition to the terminal representative (Section 4.1), with interval set directly to  $[1, 1]$ . When the image is partly inside, the representative is added as a target beside the grid targets, with interval set directly to  $[0, 1]$ ; the grid targets' intervals are computed by the factor construction as before, and no grid target is enumerated inside the region.

The provided membership test covers a region bounded in a single deterministic dimension: the region lies beyond a boundary value in that dimension, the *terminal boundary*. The dimension is deterministic, so the image box holds the actual next coordinate of every state of the source cell, and membership is read off the image box alone. Each state then either enters the region or does not, so  $[0, 1]$  is the exact range for a partly inside image. Grid targets in the bounded dimension are enumerated only up to the terminal boundary. Certain coverage is tested against the whole image, so a grid target straddling the terminal boundary receives the lower bound 0. A richer region, over several dimensions or with a stochastic threshold, requires a richer membership test; the routing itself is unchanged.

**Escape in the deterministic dimensions.** The grid models a bounded window of each dimension, and the dynamics are not confined to it. Unless floored or capped at the corresponding state-space bound, the image in a deterministic dimension can extend beyond the grid at either end. A state carried past the edge leaves the modelled window, and the safe sink of Section 4.1 is the outcome that records this. Whether an image point has left the grid follows from the half-open convention of Section 4.1. The grid's top edge is the excluded upper edge of the last cell, so a point exactly on it lies outside; the grid's bottom edge is the included lower edge of the first cell, so a point exactly on it lies inside. When the whole image has left the grid, no factors are computed, and the row is a single transition to the sink with interval set directly to  $[1, 1]$ . When part of the image leaves the grid, the sink is added as a target beside the enumerated grid targets, with interval set directly to  $[0, 1]$ , the exact range over a cell containing both escaping

and non-escaping states. One grid edge is excepted from the sink routing. In the dimension that carries the terminal region, the grid ends at the terminal boundary, and the states beyond that edge lie inside the terminal region rather than outside the model. An image crossing that edge enters the region and is routed to the terminal representative, as the absorbing definition of Section 4.1 prescribes. The sink routing is required for soundness in the sense of Section 4.3. Mass that leaves the grid must remain representable within its interval row, and a wholly escaped image with no sink target would produce an invalid IMDP.

**Product over dimensions.** The factor brackets are computed per dimension, and a transition needs one interval for the whole step. For the product kernel of (6), the step succeeds when every dimension lands in its slice of the target, so the brackets multiply, and the interval stored for the target cell with per-dimension indices  $(k_1, \dots, k_n)$  is

$$p_{\min} = \prod_{d=1}^n f_{\text{lo}}^d, \quad p_{\max} = \prod_{d=1}^n f_{\text{hi}}^d, \quad (26)$$

where

- $f_{\text{lo}}^d$  and  $f_{\text{hi}}^d$  are the ends of the factor bracket of dimension  $d$ , evaluated for the target's cell  $k_d$  in that dimension;
- $n$  is the number of dimensions.

Each index combination selects one grid cell, and the flattening of Section 4.1 assigns every grid cell a single identifier, so distinct combinations always write to distinct targets. Combinations whose upper product lies below tolerance are discarded; the discarded mass sits below the numerical tolerance of the containment certificate (Section 4.3). A lower product of 0 is stored as 0. For a target only possibly reached, some states of the source cell have true probability 0, and the stored interval must contain that value.

//here Algorithm 2 computes the image box and performs the noise check of Assumption (i). It evaluates the dynamics at every corner of the source cell and records, per dimension, the smallest and largest next value and the smallest and largest noise across the corners. The corner coordinates are the stored edge values of (9) themselves, not values reconstructed from a cell centre and a half-width. Reconstruction can differ from the stored edges in the last bits of floating-point arithmetic, and the boundary reasoning of (17) is exact only when the corners are evaluated at the delimiting values. The upper edges are among the corner coordinates, so the corner set spans the closure of the half-open cell, and the returned image interval bounds the image of every state of the cell, boundary included.

---

**Algorithm 2** IMAGEBOX: exact corner enumeration of the deterministic image, with the noise state-independence check.

---

**Require:** source-cell edges  $[\text{lo}_d, \text{hi}_d]$  for each dimension  $d$  (the exact stored values of (9)), action  $a$

**Ensure:** image interval  $[\mu^{\text{lo}}, \mu^{\text{hi}}]$  per dimension and per-cell noise  $\sigma$

```

1:  $\mu_d^{\text{lo}} \leftarrow +\infty, \mu_d^{\text{hi}} \leftarrow -\infty; \sigma_d^{\text{lo}} \leftarrow +\infty, \sigma_d^{\text{hi}} \leftarrow -\infty$  for all  $d$ 
2: for all corners  $(x_1, \dots, x_n)$  with  $x_d \in \{\text{lo}_d, \text{hi}_d\}$  (all  $2^n$  combinations of exact cell edges) do
3:    $(\mu, \sigma') \leftarrow$  deterministic next state and noise at the corner under  $a$ 
4:   for each dimension  $d$  do
5:      $\mu_d^{\text{lo}} \leftarrow \min(\mu_d^{\text{lo}}, \mu_d); \mu_d^{\text{hi}} \leftarrow \max(\mu_d^{\text{hi}}, \mu_d)$ 
6:      $\sigma_d^{\text{lo}} \leftarrow \min(\sigma_d^{\text{lo}}, \sigma'_d); \sigma_d^{\text{hi}} \leftarrow \max(\sigma_d^{\text{hi}}, \sigma'_d)$ 
7:   end for
8: end for
9: for each dimension  $d$  do
10:  if  $\sigma_d^{\text{hi}} - \sigma_d^{\text{lo}} > 10^{-12}$  then raise (noise not constant across the cell)
11:  end if
12: end for
13: return  $[\mu^{\text{lo}}, \mu^{\text{hi}}]$  and the common noise  $\sigma = \sigma^{\text{lo}}$ 

```

---

Algorithm 3 returns the extrema (20)–(21) of the landing chance over the image interval.

---

**Algorithm 3** BOXPROBRANGE: range of the landing chance over the image interval (adapting [?, Eq. 3.11]).

---

**Require:** image interval  $[\mu^{\text{lo}}, \mu^{\text{hi}}]$ , target window  $\ell^j, h^j$ , noise  $\sigma$

**Ensure:**  $(m, M)$ , the smallest and largest value of  $g$  over the image interval

```

1: if  $\sigma \leq 0$  then return  $(0, 0)$ 
2: end if
3:  $g(\mu) := \Phi((h^j - \mu)/\sigma) - \Phi((\ell^j - \mu)/\sigma); \mu^* \leftarrow (\ell^j + h^j)/2$ 
4: if  $\mu^{\text{lo}} \leq \mu^* \leq \mu^{\text{hi}}$  then  $M \leftarrow g(\mu^*)$ 
5: else  $M \leftarrow \max\{g(\mu^{\text{lo}}), g(\mu^{\text{hi}})\}$ 
6: end if
7:  $m \leftarrow \min\{g(\mu^{\text{lo}}), g(\mu^{\text{hi}})\}$ 
8: return  $(m, M)$ 

```

---

Algorithm 4 assembles one row of the IMDP. It builds the image box, applies the terminal routing and the escape routing, and computes the factor bracket of every dimension. Stochastic factors come from Algorithm 3 with the truncation division (24); deterministic factors from (17). The brackets multiply into the transition intervals (26).

---

**Algorithm 4** Per-cell, per-action transition intervals. Produces one IMDP row.

---

**Require:** source-cell edges  $[lo_d, hi_d]$  per dimension, action  $a$ , grid bins and shape

**Ensure:** interval map  $\mathcal{T} : \text{target id} \rightarrow [p_{\min}, p_{\max}]$

```

1:  $([\mu^{lo}, \mu^{hi}], \sigma) \leftarrow \text{IMAGEBOX}(lo, hi, a)$  ▷ Algorithm 2
2: if the image lies wholly inside the terminal region then return  $\{\text{term} : (1, 1)\}$ 
3: end if
4:  $\text{terminalPossible} \leftarrow$  image partly inside the terminal region
5: if the image lies wholly outside the grid in a deterministic dimension then return  $\{\text{sink} : (1, 1)\}$ 
6: end if
7:  $\text{escapePossible} \leftarrow$  image reaches the grid's top edge, or crosses below its bottom edge, in a
   deterministic dimension (terminal edge excepted)
8:  $\Pi_{lo}, \Pi_{hi} \leftarrow$  extrema of (22) over the image interval (interval ends; band midpoint for  $\Pi_{hi}$ 
   when the interval contains it)
9: for each dimension  $d$  do
10:   if  $\sigma_d > 0$  then ▷ stochastic dimension
11:     enumerate cells  $k$  within  $[\mu_d^{lo}, \mu_d^{hi}]$  extended by  $n_{\text{cut}}\sigma_d$ 
12:     per  $k$ :  $(m, M) \leftarrow \text{BOXPROBRANGE}(\mu_d^{lo}, \mu_d^{hi}, \text{bin}_{lo}^k, \text{bin}_{hi}^k, \sigma_d)$ ; divide as in (24); discard
       if  $M$  below tolerance
13:   else ▷ deterministic dimension; enumeration ends at the terminal boundary
14:     enumerate cells  $k$  the image touches; per  $k$ :  $f_{lo} \leftarrow 1$  if the image lies wholly inside the
       half-open cell  $k$ , else 0; factor  $(k, f_{lo}, 1)$ 
15:   end if
16: end for
17:  $\mathcal{T} \leftarrow \{\text{flat}(k_1, \dots, k_n) : (\prod_d f_{lo}^d, \prod_d f_{hi}^d)\}$  over all factor combinations, products below
   tolerance dropped
18: if  $\text{terminalPossible}$  then  $\mathcal{T}[\text{term}] \leftarrow (0, 1)$ 
19: end if
20: if  $\text{escapePossible}$  then  $\mathcal{T}[\text{sink}] \leftarrow (0, 1)$ 
21: end if
22: return  $\mathcal{T}$ 

```

---

### 4.3 Soundness of the abstraction

The IMDP produced by Sections 4.1 and 4.2 is intended to be verified in place of the continuous-state system. Two soundness questions arise, and they decouple. First, given the IMDP intervals, do the satisfaction probabilities computed on the IMDP correctly bracket the property value of the IMDP? Second, does the property value of the IMDP bracket the satisfaction probability on the underlying continuous system? The first is answered by the semantics of IMDP verification; the second by per-transition containment together with the preservation result of Section 2.4.

**Layer 1: robust verification is sound for the IMDP.** Given an IMDP whose every transition carries an interval  $[p_{\min}, p_{\max}]$ , a model checker such as Storm computes the worst-case ( $P_{\max}$ ) and best-case ( $P_{\min}$ ) value of the property over *all* transition distributions consistent with the stored intervals. This is a robust optimisation, not a point estimate, and the output  $[P_{\min}^{\text{Storm}}, P_{\max}^{\text{Storm}}]$  brackets the property value of the IMDP unconditionally, independently of grid resolution, of the noise model, and of how wide the per-transition intervals happen to be. Wide intervals yield wider brackets, not unsound ones.

**Layer 2: per-transition containment transfers the result to the continuous system.** The connection from the IMDP to the continuous system is the containment hypothesis of

Section 2.4. Concretely, for every source cell  $A_i$ , action  $a$ , and target cell  $A_j$  we require

$$T_x(A_j \mid s, a) \in [\check{P}(i, a, j), \hat{P}(i, a, j)] \quad \text{for every } s \in A_i. \quad (27)$$

The intervals of Section 4.2 are constructed so that (27) holds by design: the deterministic factors are exact with respect to the image box under the half-open cell convention, and the stochastic factor conservatively brackets the renormalised box probability over the image, by (25). When (27) holds, every concrete one-step transition law is a selection of the Markov set-chain that the IMDP denotes, so by [?] the robust value computed on the IMDP brackets the continuous-system value,  $p_{\text{cont}}(s_0) \in [P_{\min}^{\text{Storm}}, P_{\max}^{\text{Storm}}]$ . No additive error constant is required; the looseness of the abstraction is absorbed entirely into the width of the bracket.

**The containment certificate.** Because (27) is the sole soundness hypothesis, we verify it directly and exhaustively rather than relying on an analytic bound. The containment test samples states spanning every source cell, for every action, respecting the half-open convention of Section 4.1: in each dimension the lowest sample sits exactly on the cell’s lower edge, which the cell owns, and the highest sits at the largest floating-point value strictly below the upper edge, which it does not. At each sampled state it computes the true one-step transition probabilities to every reachable target cell (deterministic dimensions exactly, the stochastic dimension as the truncated and renormalised Gaussian, matching the modelling kernel) and checks that each lies within the stored interval to a fixed numerical tolerance, recording the excess of any violation. Running this test across the whole grid is the soundness certificate for a given configuration; the quantitative outcome on the production grid is reported in Section 5.3.

#### Scope of the soundness claim

Containment is established relative to the *modelling* kernel, the factorised kernel of Section 4.2. Any cross-dimension correlation discarded in forming that kernel (for the case study, the gap-noise correlation of Section 5.1) is deliberately discarded as part of the model. The abstraction is therefore a sound over-approximation of the modelled stochastic system; it is not a claim about a fully-correlated physical kernel that the model does not represent.

**The additive bound is a diagnostic only.** For comparison with [?] we also compute the additive global error bound. Define the per-row half-width sum and the additive bound

$$K(i, a) = \sum_j \frac{1}{2} (\hat{P}(i, a, j) - \check{P}(i, a, j)), \quad E = N \cdot \max_{i,a} K(i, a), \quad (28)$$

with  $N$  the verification horizon; the pipeline reports  $E$  at  $N = 10$ , a fixed reporting convention independent of the verification horizon used in Section 5.3. This quantity is reported only as a description of the abstraction’s interval widths. It is *not* the soundness mechanism, and it is structurally uninformative for this system: the only stochastic dimension carries constant noise, so  $K$  is essentially spatially uniform, the additive bound concentrates nowhere that adaptive refinement could target, and on the production grid  $E$  exceeds 1 at the practically interesting horizons. The soundness of the abstraction does not depend on  $E$  being small; that is the role of the containment certificate, which holds regardless of the value of  $E$ . Algorithm 5 computes  $E$  from the assembled IMDP.

---

**Algorithm 5** GLOBALERRORDIAGNOSTIC: descriptive interval-width summary; not the soundness certificate.

---

**Require:** IMDP  $\mathcal{I}$  with intervals, horizon  $N$

- 1: for each  $(i, a)$ , set  $K(i, a) \leftarrow \sum_j \frac{1}{2} (\hat{P}(i, a, j) - \check{P}(i, a, j))$
  - 2:  $(i^*, a^*) \leftarrow \arg \max_{i,a} K(i, a)$ ;  $E \leftarrow N \cdot K(i^*, a^*)$
  - 3: **return**  $(E, i^*, a^*)$
-

#### 4.4 The full pipeline

The end-to-end procedure composes the steps above. An injected lead-behaviour model supplies the exogenous lead rule, the noise level, and the grid band in the stochastic dimension. The orchestrator builds the grid; for each source cell and action it computes the row of per-transition intervals via Algorithm 4; it finalises the safe-sink state; it computes the diagnostic bound (Algorithm 5); it attaches per-cell controller actions and atomic-proposition labels, both read at cell centres; and it emits a PRISM model encoding the plant IMDP, the perception layer, the controller, and the labelling, together with a second encoding of the same abstraction, described in Section 5.3, used for model checking at the production scale. The discretisation step is parallelised over chunks of source cells; completed rows are accumulated in memory and both encodings are stream-written to file at export. At the production grid size the accumulated transition structure exceeds the memory of a development workstation, so production generation is performed on a large-memory cluster node (Section 5.3). Algorithm 6 states the procedure.

---

**Algorithm 6** End-to-end IMDP abstraction pipeline.

---

**Require:** plant  $P$ , controller  $C$ , lead model  $L$ , grid resolutions  $\{r_d\}$ , horizon  $N$

**Ensure:** PRISM model encoding the IMDP abstraction, the closed-loop encoding of Section 5.3, and the diagnostic  $E$

```

1: resolve  $L$ ; pin the stochastic-dimension band and noise from  $L$ ; reject  $L$  if nondeterministic
2: build the grid (Algorithm 1), obtaining  $M$  cells and a cell-centre map
3: initialise an empty IMDP  $\mathcal{I}$ ; mark the terminal cell absorbing under every action
4: for each source cell  $i \in \{1, \dots, M-1\}$  in parallel chunks do
5:   for each action  $a$  do
6:      $\mathcal{T} \leftarrow$  per-cell intervals for the cell's edges  $(lo_i, hi_i)$  under  $a$  ▷ Algorithm 4
7:     store  $\mathcal{T}$  as the row  $(i, a)$  of  $\mathcal{I}$ 
8:   end for
9: end for
10: mark cell  $M$  (safe sink) as an absorbing self-loop under every action
11:  $(E, i^*, a^*) \leftarrow \text{GLOBALERRORDIAGNOSTIC}(\mathcal{I}, N)$  ▷ diagnostic, Section 4.3
12: for each source cell  $i$  do
13:   record the controller action  $C(c_i)$  as a PRISM controller rule
14:   attach atomic-proposition labels to  $i$  according to the region containing its centre  $c_i$ 
15: end for
16: emit the PRISM model: plant IMDP module, perception module, controller module, and labels
17: emit the closed-loop encoding of the same abstraction (Section 5.3)
18: return PRISM model, closed-loop encoding,  $E$ ,  $(i^*, a^*)$ 

```

---

*Remark.* The controller rule and the atomic-proposition labels of a cell are read at its centre. A cell that straddles a controller switching threshold, or a label boundary, is therefore assigned the action and the labels of its centre throughout its extent. The object verified is thus the controller as quantised on the grid, which is exactly the controller the emitted PRISM model executes, rather than the continuous-state controller; the plant intervals are unaffected. Refining the grid shrinks the straddling cells and brings the quantised controller closer to the continuous one.

The emitted PRISM model consists of four modules synchronised by a small scheduler that enforces the order `perceive`  $\rightarrow$  `control`  $\rightarrow$  `time_step`. The plant module encodes the IMDP intervals; the perception module is the identity map in the perfect-perception setting (and an explicit enumeration of perception outputs in a noisy setting); the controller module is run-length-encoded over contiguous state ranges sharing an action; and the labels (`crash`, `safe`, `nominal`, `braking`, `emergency`, `safe_physics`, `safe_sink`) make the case-study scenarios and regions addressable by the verifier.

## 4.5 Adaptation of, and departure from, the partition-based framework

Our construction reuses one analytic ingredient from [?] and departs from it in several deliberate ways. The ingredient we adopt is the per-transition range bound of [?, Eq. 3.11]: for the single stochastic dimension we bracket the cell-to-cell probability by the exact range of the integrated Gaussian over the image of the source cell, which Algorithm 3 computes in closed form. We do not adopt the adaptive gridding procedure of [?, Algs. 3–4]; because the only stochastic dimension carries constant noise, the per-row interval width is essentially uniform across the state space and presents no localised error for refinement to exploit, so a uniform grid is both simpler and, here, no less informative.

The most consequential departure is on the deterministic dimensions. Rather than collapsing a source cell to the image of its centre, which is the representative-point step of a point-probability construction, we enumerate the exact image of the whole cell by corner evaluation (Algorithm 2) and bracket every target cell the image touches, contributing a certain factor where the image lies wholly inside a target cell and the interval  $[0, 1]$  where it straddles a boundary (17). This is what makes the construction an over-approximation in the sense required by (27): an earlier centre-routing version of our own kernel placed all of a cell’s mass in the single target containing the centre’s image and was not sound near cell boundaries.

Soundness itself is not inherited from the additive error formula of [?, Eq. 3.13], which we compute only as the diagnostic (28) and which is in any case inapplicable to the deterministic dimensions, whose indicator kernels admit no density (Section 3), but from the containment hypothesis (27) and the robust-PCTL preservation result of [?] over Markov set-chains [?]; the underlying notion of approximate abstraction is that of [?]. Finally, the construction is split into separate plant, perception, and controller modules. This architectural decision does not bear on soundness, but it enables the later composition with a separately-constructed perception abstraction described in Section 5.7, and it keeps the controller as an explicit deterministic policy over the abstract states, quantised at cell centres (see the remark in Section 4.4), rather than folding it into the plant abstraction. The split has a further consequence. The plant rows are indexed by abstract state and commanded input alone, and are therefore independent of the control logic that selected the input. A controller with persistent internal modes, a supervisor that switches between control laws at run time, is therefore just a larger deterministic controller module: a discrete mode variable with symbolic switching rules alongside the action rules already emitted. It would compose with the same plant IMDP without any change to the abstraction pipeline, to the containment certificate, or to the transfer argument, since the preservation result of [?] concerns finite interval models and is indifferent to how their state space is structured. The modular architecture thus extends the construction to hybrid control logic, discrete modes co-evolving with the continuous state, at no cost to the plant abstraction; Section 5.1 notes what this means for the case study’s classification.

## 4.6 Choosing the partition resolution

The brackets a resolution can deliver depend on the property’s horizon. This subsection states the quantitative rules by which a resolution is matched to a family of properties. The rules are heuristics. They predict where brackets saturate; Sections 5.4 and 5.5 report the measurements that test them.

**The per-period displacement budget.** In a deterministic dimension, a target straddled by the image carries the lower bound 0 (17), and the row’s upper bounds confine every admissible distribution to the block of cells the image box touches. The worst-case resolution of the intervals can therefore displace the deterministic coordinates by up to one cell width per control period around the modelled dynamics, in a direction of its choosing. The displacements renew each period: the reachable set of the interval model grows by a fixed increment per step, independently of where earlier mass landed. This renewed-per-step growth is the wrapping effect of interval analysis [WRAPPING-EFFECT REFERENCE: TO BE READ AND CITED BEFORE CIRCULATION];

the additive diagnostic  $E = N \cdot \max K$  of (28) is its linear expression. For state-independent noise the per-row half-width sum is spatially uniform: the assembled models report the identical worst value  $K = 6.276171$  on two grids of different resolution (Sections 5.3 and 5.5), at different worst states.

**The crossing horizon.** For a start state at distance  $D$  from the terminal region along the gap axis, the budget predicts the *crossing horizon*: the first queried step bound at which  $P_{\max}$  leaves 0,

$$k_{\text{cross}} \approx \frac{D}{(v_{\text{ego}} - v_{\text{lead}}^{\min}) \Delta t + r_g}, \quad (29)$$

where

- $k_{\text{cross}}$  is the predicted crossing horizon, in control periods;
- $D$  is the gap distance from the start state to the terminal region;
- $v_{\text{ego}} - v_{\text{lead}}^{\min}$  is the largest closing speed the model admits: the ego speed against the lowest lead speed of the grid band;
- $r_g$  is the gap resolution, contributing one cell of displacement budget per period.

Below  $k_{\text{cross}}$  the worst case cannot reach the terminal region and  $P_{\max} = 0$  is certified; above it,  $P_{\max}$  rises towards saturation at 1. Equation (29) is a prediction heuristic, not a bound. Its measured status: the predicted crossings match the measured values at five crossings across two sweep grids, and the Emergency Brake crossing is later than predicted, so the heuristic errs on the conservative side (Section 5.5).

**Design rules.** Four rules summarise the measured behaviour; each is stated with its evidence.

1. *Horizon budget.* A property at step bound  $k$  is decidable only if  $k$  lies below the crossing horizon (29) of its start state. Evidence: the frontier of Section 5.4 and the sweep crossings of Section 5.5.
2. *Control visibility.* The ego resolution must satisfy  $r_{v_{\text{ego}}} < \alpha|a|\Delta t$  for a braking action  $a$  to be expressible on the grid: one period of braking must move the ego image by at least one cell. For the case study the thresholds are 0.2 m/s (brake) and 0.49 m/s (emergency). The production resolution  $r_{v_{\text{ego}}} = 0.5$  fails both; the sweep probe at 0.25 passes the emergency threshold only; the second certified configuration at 0.1 passes both. Evidence: the probe grid's fractional certificate at  $k = 52$  (Section 5.5).
3. *Stochastic non-degeneracy.* On a stochastic dimension the ratio  $\sigma_d/r_d$  must remain of order one or above, or the range factor (??) degenerates. The reported configurations use  $\sigma_v/r_{v_{\text{lead}}} \approx 2.0$ .
4. *Cell budget.* The cell count  $M = \prod_d(\text{range}_d/r_d)$  is capped by generation memory and disk staging, approximately  $2 \times 10^6$  cells on the reported infrastructure. A finer resolution is therefore afforded by shrinking the window; Section 5.6 instantiates the trade.

## 5 Case Study: An Autonomous Emergency Braking System

We apply the procedure of Section 4 to a three-dimensional model of an Autonomous Emergency Braking System. Section 5.1 describes the continuous system, Section 5.2 the implementation architecture, Section 5.3 the experimental configuration, the certificates, and the ten-second campaign, Section 5.4 the horizon-resolved campaign, Section 5.5 the resolution sweep, Section 5.6 the second certified configuration and its results, and Section 5.7 the future work.

## 5.1 System description

**State space.** The continuous state is the triple

$$s = (g, v_{\text{ego}}, v_{\text{lead}}) \in \mathcal{S} = [0, 100] \times [0, 30] \times [-1, 31] \subset \mathbb{R}^3, \quad (30)$$

where  $g$  is the relative gap (m),  $v_{\text{ego}}$  the ego velocity (m/s), and  $v_{\text{lead}}$  the lead velocity (m/s). The lower bound  $-1$  on  $v_{\text{lead}}$  provides one cell of headroom below zero so the discretised Gaussian tail at  $v_{\text{lead}} = 0$  does not immediately escape the grid; physically  $v_{\text{lead}} \geq 0$ . The remaining bounds are likewise modelling choices, sized to envelope the scenarios of Section 5.3: velocities up to 30 m/s (108 km/h) exceed every speed exercised there, and an initial gap of up to 100 m places even the fastest approach several seconds from its lead.

**Actions.** The controller commands one of three accelerations,

$$U = \{a_0 = 0, a_1 = -4, a_2 = -9.8\} \text{ m/s}^2, \quad (31)$$

labelled *coast*, *brake*, and *emergency*.

**Controller logic.** The controller selects among the actions of (31) as a deterministic, memoryless function of the perceived state, through two derived quantities: the closing speed  $v_{\text{rel}} = v_{\text{ego}} - v_{\text{lead}}$  and the time to collision  $\text{TTC} = g/v_{\text{rel}}$ , the latter defined when  $v_{\text{rel}} > 0.1$  m/s and taken as unbounded otherwise [?]. Its thresholds and decelerations are those of the implementation’s `industry` configuration [CITATION FOR THE INDUSTRY BENCHMARK THRESHOLDS]. At low closing speeds ( $v_{\text{rel}} < 5$  m/s) the decision is distance-based: *emergency* ( $a_2$ ) when  $g < 2$  m, *brake* ( $a_1$ ) when  $2 \leq g < 6$  m, and *coast* ( $a_0$ ) otherwise. At higher closing speeds it is TTC-based: *emergency* when  $\text{TTC} < 1.0$  s, *brake* when  $1.0 \leq \text{TTC} < 1.6$  s, and *coast* otherwise. An early-braking variant of the controller (`safe`), with wider thresholds and an emergency deceleration of  $-8$  m/s<sup>2</sup>, is retained in the implementation for comparison runs and is not reported here. In the abstraction the deployed function is evaluated at cell centres and emitted as the run-length-encoded controller module of Section 4.4.

**Terminal region and atomic propositions.** The designated terminal region of Section 4.1 is  $g \leq 0$ : transitions whose image reaches it are routed to the absorbing crash cell by Algorithm 4. The labels emitted with the model are predicates over the same derived quantities as the controller, evaluated at cell centres. `crash` holds on the cells whose centres satisfy  $g \leq 0$ , that is, on the full slab of cells at the lowest gap index and not only on the absorbing cell itself; `safe` holds on every other grid cell, and `safe_sink` on the safe sink. Two further labels mark dual-trigger risk regions, bound to the controller’s warning and emergency thresholds: `braking` holds where  $g < 10$  m or  $\text{TTC} < 2.6$  s, and `emergency` where  $g < 2$  m or  $\text{TTC} < 1.0$  s. The label `nominal` holds on the `safe` cells outside `braking`: the states in which no braking is called for. Neither risk region coincides with the set of cells on which the controller commands the corresponding action, for two reasons: the `braking` thresholds are the warning thresholds, wider than the braking thresholds of the decision logic, and the decision logic branches on the closing speed whereas the labels apply both triggers at every state. A property over `braking` is therefore a statement about the *warning region*, not about the controller braking, and a property over `emergency` is a statement about the *emergency region*, not about the controller emergency-braking. The label `safe_physics` is at present identical to `safe` and is retained for interface compatibility.

**One-step deterministic dynamics.** The plant computes a deterministic next state given  $(s, a)$ , split into an ego update, an exogenous lead update, and a gap update,

$$a_{\text{ego}} = \alpha \cdot a, \quad \alpha = 0.5 \text{ (static actuator gain)}, \quad (32)$$

$$v'_{\text{ego}} = \max\{0, v_{\text{ego}} + a_{\text{ego}} \cdot \Delta t\}, \quad (33)$$

$$v'_{\text{lead}} = \max\{0, v_{\text{lead}}\} \quad (\text{default lead rule, overridable by an injected model}), \quad (34)$$

$$g' = g + (v'_{\text{lead}} - v'_{\text{ego}}) \cdot \Delta t, \quad (35)$$

with  $\Delta t = 0.1$  s, the modelled decision period of the controller. Equation (35) uses the updated velocities  $v'_{\text{ego}}$  and  $v'_{\text{lead}}$  rather than their previous-step values, making this a semi-implicit (symplectic) Euler step. The ego update is controller-driven; the lead update is exogenous and does not run the controller. The factor  $\alpha$  attenuates the commanded acceleration as a memoryless gain; the simulation environment implements a full first-order actuator lag, of which this stateless gain is the abstracted simplification; the value  $\alpha = 0.5$  is itself a modelling choice, representing a conservatively weak actuator response.

**Monotonicity and continuity.** Assumption 1(iii) requires each output coordinate of the one-step map to be monotone in each input coordinate, and (33)–(35) satisfy this.  $v'_{\text{ego}}$  is nondecreasing in  $v_{\text{ego}}$  and independent of the other coordinates, the flooring  $\max\{0, \cdot\}$  preserving monotonicity;  $v'_{\text{lead}}$  is nondecreasing in  $v_{\text{lead}}$  and independent of the others, for the same reason; and  $g'$  is nondecreasing in  $g$ , nondecreasing in  $v_{\text{lead}}$  through  $v'_{\text{lead}}$ , and nonincreasing in  $v_{\text{ego}}$  through  $v'_{\text{ego}}$ . The extrema of every output coordinate over a cell are therefore attained at the cell's corners, as (15) requires. Each output coordinate is moreover continuous in the state, the floors  $\max\{0, \cdot\}$  being continuous functions, which discharges the continuity requirement of Assumption 1(ii). The same floors settle the lower-boundary escape case of Section 4.2 for this system:  $v'_{\text{ego}}$  cannot fall below zero, and zero lies in a cell interior by the half-cell offset of (9), so bottom-of-grid escape does not arise in  $v_{\text{ego}}$ ; the gap's lower boundary is the crash wall, handled by the terminal routing; and out-of-band mass in the stochastic dimension is handled by the truncation policy, not by escape routing.

**Stochastic model.** The kernel is Gaussian as a consequence of the following modelling assumption.

**Assumption 2** (Lead-vehicle uncertainty). *The lead-vehicle acceleration is an i.i.d. Gaussian random variable,  $a_{\text{lead}}^{(k)} \sim \mathcal{N}(0, \sigma_a^2)$ , independent across time steps. The ego dynamics and the gap update are deterministic given  $(s, a)$  and the realisation of  $a_{\text{lead}}^{(k)}$ .*

The constant  $\sigma_a$  aggregates a small plant-process noise and the lead-acceleration noise,

$$\sigma_a = \sqrt{\sigma_{\text{plant}}^2 + \sigma_{\text{lead}}^2} = \sqrt{0.05^2 + 2.0^2} \approx 2.0006 \text{ m/s}^2, \quad (36)$$

and a first-order Euler step on velocity gives the per-step velocity noise and the lead-velocity kernel

$$\sigma_v = \sigma_a \cdot \Delta t \approx 0.2001 \text{ m/s}, \quad v'_{\text{lead}} \sim \mathcal{N}(\max(0, v_{\text{lead}}), \sigma_v^2). \quad (37)$$

The numerical values are modelling choices rather than standardised quantities:  $\sigma_{\text{lead}} = 2.0 \text{ m/s}^2$  is chosen so that, through (37), the lead's step-to-step speed change is moderate (a standard deviation of 0.2 m/s per 0.1 s step), and  $\sigma_{\text{plant}} = 0.05 \text{ m/s}^2$  is a small allowance for unmodelled plant disturbance; neither is calibrated against measured driving data at this stage of the programme.

*Remark* (Process noise is prediction uncertainty, not estimation uncertainty). The Gaussian term in (37) models what the lead vehicle will do over the coming time step, not how accurately the current state is measured. The kernel mean  $\max(0, v_{\text{lead}})$  is the lead model’s deterministic prediction of the next lead velocity, and the scatter around it is the effect of the lead acceleration of Assumption 2, which at the moment of prediction has not yet been realised. Under the perfect-perception setting of this paper the current state  $(g, v_{\text{ego}}, v_{\text{lead}})$  is known exactly; the uncertainty concerns its evolution, and no sensor fidelity reduces it, since a sensor reports the lead’s present velocity rather than its driver’s next decision. It is therefore distinct from, and orthogonal to, the perception uncertainty of the later tracks (Section 5.7), which concerns errors in estimating the current state; the perception-aware construction composes the two rather than substituting one for the other.

**The gap-noise approximation.** Since  $g' = g + (v'_{\text{lead}} - v'_{\text{ego}}) \Delta t$  and  $v'_{\text{lead}}$  is the only random input,  $g'$  inherits noise from  $v'_{\text{lead}}$ : the true joint distribution of  $(g', v'_{\text{lead}})$  is a degenerate (rank-one) bivariate Gaussian, with

$$\sigma_g = \sigma_v \cdot \Delta t \approx 0.02 \text{ m}. \quad (38)$$

We discard this correlation and treat  $g$  as deterministic given the centroid. This is a deliberate simplification: with cell width of order 1 m and  $\sigma_g \approx 0.02 \text{ m}$ , a single Gaussian step in gap is well within one cell, and the simplification keeps the kernel factorisable as a product of marginals across dimensions. The implementation reflects this by setting  $\sigma_{\text{per dim}} = (0, 0, \sigma_v)$ . The containment certificate of Section 4.3 is evaluated against this same modelling kernel.

**Kernel structure.** The modelling kernel is therefore a product of three one-dimensional factors, discharging the factorisation requirement of Assumption 1(i): Dirac point masses on gap and  $v_{\text{ego}}$ , whose one-step values are computed exactly by (33) and (35), and a Gaussian density on  $v_{\text{lead}}$ , given by (37). Figure 15 traces one step: the transition probability to a target cell is the product of two indicator factors and one area factor. This is the concrete instance of the generic factor split of Section 4.2 (Figure 14), and it delimits where the ingredients of Section 4.5 apply: the range bound of [?] operates on the density factor, and the image-box enumeration on the two point-mass factors.

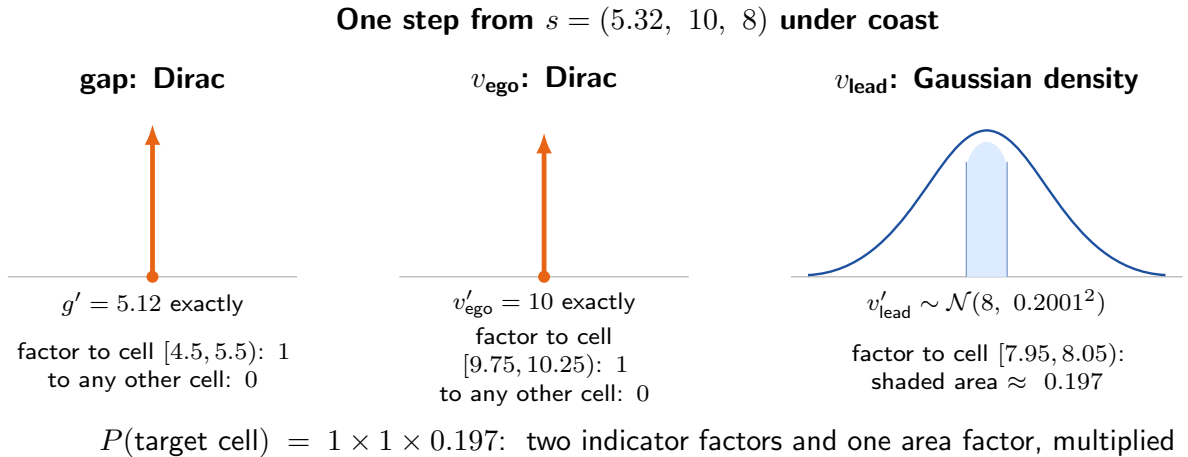


Figure 15: The AEBS modelling kernel as a product of three one-dimensional factors, traced for one step. The deterministic dimensions carry Dirac point masses (indicator factors); the lead velocity carries a Gaussian density (an area factor). The probabilities shown are exact for the `medium_tight` grid of Section 5.3.

**Lead-behaviour model.** The exogenous lead rule of (34) is supplied by an injectable lead-behaviour model, which owns the lead’s dynamics and their modelling envelope: the transition rule, the noise level of Assumption 2, and the grid band of the stochastic dimension. One deterministic profile is composed at present, implementing the hold rule of (34): the lead’s next velocity is its current velocity, floored at zero. A stationary lead and a constant-speed lead are both instances of this rule, distinguished only by the initial lead velocity; both correspond to standard car-to-car rear test scenarios [?]. Verification scenarios are not properties of the lead model: they are start states of the encounter, specified by the ego’s situation at time zero in continuous coordinates and resolved to grid cells (Section 5.3), so one scenario catalogue serves any lead profile whose grid band contains its states. With a moving lead the gap update couples to the lead velocity, which shears the image set of each source cell and widens the deterministic gap bracket relative to the stationary case; this affects the tightness of the intervals, not their soundness. A nondeterministic lead, which may apply any of a set of admissible accelerations at each step [?], is defined in the implementation but not composed: it requires a second (lead) action axis in the discretiser and the composition, and its treatment is deferred to Section 5.7.

**Formal classification.** With a deterministic lead, the system is a discrete-time stochastic control system  $\Sigma = (\mathcal{X}, U, T_x)$  in the sense of [?, Sec. 3], with  $\mathcal{X}$  continuous,  $U$  finite, and  $T_x$  a Dirac–Gaussian product kernel; the controller’s operating regimes are values of the input  $a \in U$ , not discrete plant modes, so  $|Q| = 1$  and the full hybrid-mode machinery of [?, Sec. 4] is not needed. The restriction to  $|Q| = 1$  is a property of the present closed loop, not of the construction: because the plant rows depend only on the cell and the commanded acceleration, a controller with a persistent internal mode, for instance a supervisor toggling between the deployed law and a fallback law, would make the composed system a stochastic hybrid system with  $|Q| \geq 2$  while reusing this plant IMDP and its containment certificate unchanged (Section 4.5). The runtime monitors envisaged by the wider programme, which may switch to a fallback law on detecting a perception failure, are supervisors of exactly this kind. A nondeterministic lead would introduce a second action axis, over which the model checker would quantify in the worst case; building that path is the deferred extension.

## 5.2 Implementation architecture

The case-study system comprises four logical subsystems (Figure 16): two perception components that consume camera and radar inputs, a controller that fuses their outputs and selects an action, and a vehicle plant whose continuous dynamics produce the next state. Under perfect perception the perception components are exact, and only the plant is abstracted; the perception and controller subsystems are exact symbolic translations into PRISM.

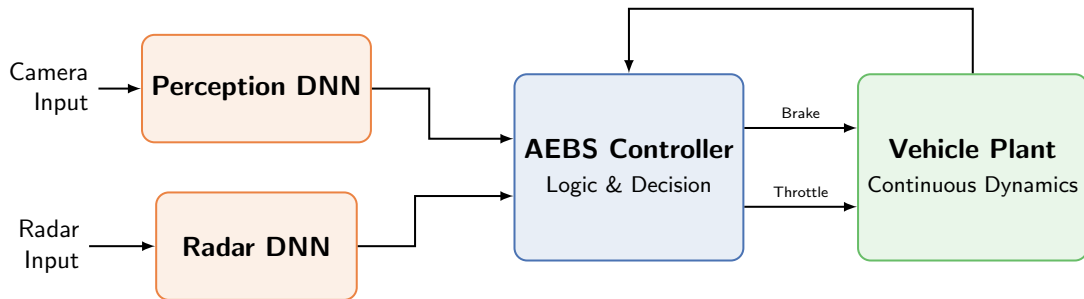


Figure 16: Case-study system architecture. Under perfect perception only the plant is abstracted.

The PRISM encoding comprises four modules, synchronised by a small scheduler.

| Module                     | Responsibility                                                                                                 |
|----------------------------|----------------------------------------------------------------------------------------------------------------|
| Plant (IMDP)               | Stochastic physics; built by discretisation.                                                                   |
| Perception (deterministic) | Identity in perfect perception; optional sensor noise otherwise.                                               |
| Controller (deterministic) | Symbolic action selection; one rule per cell, run-length encoded.                                              |
| Turn (scheduler)           | Enforces order <code>perceive</code> $\rightarrow$ <code>control</code> $\rightarrow$ <code>time_step</code> . |

### 5.3 Experimental configuration, certificates, and verification

**Configuration.** We report the `medium_tight` grid preset, the configuration on which the abstraction is certified, with the deterministic lead started stationary. The grid shape is (101, 61, 321), giving  $M = 1,977,681$  cells.

| Parameter                                                        | Value                                                  |
|------------------------------------------------------------------|--------------------------------------------------------|
| Grid resolution $(r_g, r_{v_{\text{ego}}}, r_{v_{\text{lead}}})$ | (1.0, 0.5, 0.1)                                        |
| Bounds $(g, v_{\text{ego}}, v_{\text{lead}})$                    | $[0, 100] \times [0, 30] \times [-1, 31]$              |
| Grid shape $(n_g, n_{v_{\text{ego}}}, n_{v_{\text{lead}}})$      | (101, 61, 321)                                         |
| Number of cells $M$                                              | 1,977,681                                              |
| Lead-acceleration std $\sigma_a$                                 | $\approx 2.0006 \text{ m/s}^2$ (Eq. 36)                |
| Velocity-step std $\sigma_v$                                     | $\approx 0.2001 \text{ m/s}$ (Eq. 37)                  |
| Time step $\Delta t$                                             | 0.1 s                                                  |
| Action set $U$                                                   | $\{0, -4, -9.8\} \text{ m/s}^2$ (Eq. 31)               |
| Controller configuration                                         | <code>industry</code>                                  |
| Lead model                                                       | hold rule, deterministic [?]                           |
| Stochastic-tail policy                                           | truncating; renormalised within the grid band (Eq. 24) |

The state count reflects the choice of  $r_{v_{\text{lead}}} = 0.1 \text{ m/s}$ , which is half the velocity-step noise standard deviation  $\sigma_v$ , giving a noise-to-cell-width ratio of  $\sigma_v/r_{v_{\text{lead}}} \approx 2.0$ . This is at the lower end of what keeps the range-based factor (??) non-degenerate; the resolution is essentially as coarse as it can be taken without the stochastic factor collapsing.

**Containment certificate.** The containment test of Section 4.3 is run over the entire `medium_tight` grid. It visits every one of the  $M = 1,977,681$  source cells under all three actions and samples 3 states per dimension, 27 per cell, spanning the half-open extent of the cell: in each dimension the samples run from the exact lower edge to the largest floating-point value strictly below the upper edge. At each sampled state it checks that the true one-step transition probability to each reachable target cell lies within the stored interval, at a numerical tolerance of  $10^{-7}$ . The tolerance dominates the only mass the construction deliberately neglects, the Gaussian tail beyond the  $\pm 6\sigma_v$  search window and the dropped negligible products of Section 4.2, both of order  $10^{-9}$  or below. Certificates are bound to a *run identity* comprising the lead model, the grid preset, the partition fingerprint (a hash of the exact cell edges; here 3672963e58eb75b5), a hash of the kernel source, the controller configuration together with its action values, and the test version; progress recorded under one identity cannot be resumed under another, so a certificate can never silently mix configurations. For the configuration of this section the test visited all 1,977,681 cells under all three actions, processed in 989 chunks on 32 cores of the cluster node, and completed with *zero* violations and a worst recorded excess of 0; the run identity is `static_industry_medium_tight_g3672963e_ka6a68d63_abb83a370_v3`, at code revision 3b9f3e9, and the run completed in under 65 minutes of wall-clock time. This discharges the containment hypothesis (27) for the reported configuration and, together with the robust-verification argument of Section 4.3, certifies the abstraction as a sound over-approximation of the modelling kernel.

**Interval-width diagnostic.** The diagnostic (28) was computed on the assembled production IMDP: the worst-row half-width sum is  $K = 6.276171$ , giving  $E = N \cdot K = 62.761707$  at the reporting horizon  $N = 10$ , attained by a *coast* row in the topmost  $v_{\text{lead}}$  band of the grid. A bound of this magnitude on a probability is vacuous, as anticipated structurally in Sections 3 and 4.3: with state-independent noise the per-row width is essentially uniform across the state space, and the maximising rows sit where the truncation renormalisation of (24) divides by the smallest in-band Gaussian mass, at the edge of the grid band. The containment certificate above is unaffected;  $E$  plays no role in it.

**Structure of the interval lower bounds.** The deterministic factor (17) contributes its certain value only when the image interval lies strictly inside a single half-open target cell, and for this system that containment never holds on an interior cell. In the gap dimension the image of a cell is the cell’s own width plus the shear contributed by the velocity coupling of (35), so it is strictly wider than one cell; in the  $v_{\text{ego}}$  dimension the image is a translation of the cell by  $a_{\text{ego}} \cdot \Delta t$ , of exactly the cell’s width, which fails the strict upper inequality of (17) even when the translation aligns with the grid. Every interior transition therefore carries the lower bound 0, at every grid resolution; refinement does not remove this. Each such bound is exact rather than conservative: for a straddled target, some state of the source cell reaches it with probability 0, so 0 is the true minimum of (11) over the cell. The looseness of the abstraction lies instead in the independence of the per-target intervals: the stored constraints bound each target separately, together with row feasibility, and discard the fact that all states of a cell follow a single one-step distribution. The effect on the verified values is bounded by the row’s upper bounds, which confine every admissible distribution to the block of target cells the image box touches; the freedom the zero lower bounds grant therefore amounts to a displacement of at most one cell width per step around the modelled dynamics. Refining the grid shrinks that displacement in physical units and narrows the verified brackets, although the per-transition zeros persist. The pair  $[P_{\min}, P_{\max}]$  is reported as a bracket whose width is the abstraction’s precision at the property’s horizon: the displacement budget grows by one cell width per period, so the brackets are informative only at horizons over which the accumulated budget remains small against the distances that decide the property.

**Model generation.** Generating the model at this resolution exceeds the memory of the development workstation: the image-box kernel produces several times more transitions per row than a single-target routing would. Production generation is therefore performed on a departmental cluster node with 64 cores and 503 GB of memory, parallelised over 63 worker processes; the partition fingerprint recorded at generation is required to match that of the containment certificate. Both exporters print every interval bound to eight decimal places, rounded outward: lower bounds are rounded down and upper bounds up, clamped to  $[0, 1]$ . Outward rounding widens a printed interval by at most one quantum of  $10^{-8}$ , and a widened interval that contained the true transition probability still contains it, so the printed model inherits the containment property of the in-memory intervals; the quantisation is two orders of magnitude below the certificate tolerance. For the configuration of this section, generation completed in approximately 43 minutes of wall-clock time, producing a modular PRISM artifact of 31,073,746,675 bytes (approximately 28.9 GiB) and a direct-encoding artifact of 11,002,523,700 bytes (approximately 10.2 GiB), at code revision 3b9f3e9.

**Model checking at scale.** Storm’s PRISM engine parses a model file into an in-memory syntactic representation before constructing the state space, and at the size of the production artifact that representation exceeded the memory of the cluster node: seven per-scenario invocations against the modular PRISM file were terminated by the operating system after 2.5 to 3.4 hours each, returning no results. The pipeline therefore emits, alongside the modular

PRISM model, the same abstraction in Storm’s *direct encoding* (DRN) [?]: an explicit listing of states, actions, interval transitions, and state labels that the checker streams directly into its sparse model representation, with no syntactic intermediate. The direct encoding has no module or synchronisation syntax, so the object emitted in it is the flattened synchronised product of the modular model, which is in any case the object Storm itself constructs when checking the modular file.

Under perfect perception the product admits an exact reduction. Write  $C(c_i)$  for the quantised controller of Section 4.4, the action assigned to state  $i$  at its centre. In the modular model one *control period*, of duration  $\Delta t$ , consists of three synchronised transitions: **perceive** copies the plant state to the controller’s input, **control** applies the controller function to that input, and **time.step** draws the next plant state from the row of the current state under the commanded action. The first two transitions are deterministic, so the action commanded at plant state  $i$  is  $C(c_i)$  in every period, and the product reduces to the *closed-loop* IMDP over the plant’s own states, carrying a single action per state:

$$\hat{U}_{cl}(i) = \{C(c_i)\}, \quad \check{P}_{cl}(i, \cdot) = \check{P}(i, C(c_i), \cdot), \quad \hat{P}_{cl}(i, \cdot) = \hat{P}(i, C(c_i), \cdot), \quad (39)$$

where  $\check{P}, \hat{P}$  are the plant intervals of (3) and the subscript marks the closed-loop model. The reduction changes the step accounting: one transition of the closed-loop encoding is one control period, whereas one period costs three transitions of the modular model. A bounded-reachability horizon of  $N$  periods is therefore expressed as a step bound of  $N$  on the closed-loop encoding and of  $3N$  on the modular model; property step bounds are not interchangeable between the two.

**Equivalence certificate.** The closed-loop encoding is produced by a separate exporter, so its agreement with the modular model is checked file against file rather than assumed. The check first parses the modular file’s perception module and fails unless it is the symbolic identity, the validity condition of the reduction (39); it then parses the run-length-encoded controller module into the full map  $i \mapsto C(c_i)$ . It then verifies, streaming both files in lockstep: that the closed-loop encoding carries exactly one action per state; that this action equals  $C(c_i)$ ; that its transition list equals the plant row of  $(i, C(c_i))$  in the modular file, target for target and bound for bound; and that the label state sets of the two files coincide. A cryptographic hash of the compared transition stream is recorded on both sides as the certificate of a passing run. The reduction is additionally validated numerically: on a coarse grid of 11,088 cells, the bounded-reachability values of the modular model at step bound  $3k$  and of the closed-loop encoding at step bound  $k$  agree to within  $10^{-12}$  for  $k$  up to 100, under both the worst-case and the best-case interval resolution. For the production artifacts the check passed: the direct encoding carries exactly one action per state across all 1,977,682 states (the grid cells and the safe sink), every transition list matches the controller-selected plant row bound for bound, and the label state sets of the two files coincide, with the scenario labels present only in the direct encoding as intended, in approximately 31 minutes of wall-clock time; the canonical hash 052370659bd5733e512f2175a93f7d075b8faa2751232452adfa1d21d45fe6c3 was recorded identically for the composed modular model and the direct encoding.

**Scenario states.** The verification campaign addresses four non-absorbing scenario states, staged points along a car-to-car rear encounter with a stationary lead (the CCR scenario family of [?]), together with one state inside the terminal region used as a sanity check; Table 1 lists them. The scenarios are specified by the ego’s situation at time zero, in continuous coordinates, and each resolves to the cell of the `medium_tight` grid whose centre is the stated state; the resolved identifiers are functions of the partition fingerprint alone. The action column records the controller decision at the cell centre under the logic of Section 5.1; the scenario names describe the staging of the encounter, not that decision. In the closed-loop encoding each scenario state additionally carries a dedicated state label, and the region labels of Section 5.1 are carried by

both encodings, so that scenarios and regions are addressable by the verifier without external state identifiers.

| Scenario           | $(g, v_{\text{ego}}, v_{\text{lead}})$ | Action            | State id |
|--------------------|----------------------------------------|-------------------|----------|
| Safe_Cruising      | (100, 15, 0)                           | coast             | 1967740  |
| Warning_Brake      | (30, 7, 0)                             | coast             | 591934   |
| Emergency_Brake    | (10, 10, 0)                            | brake             | 202240   |
| Imminent_Collision | (5, 30, 0)                             | emergency         | 117175   |
| Post_Collision     | (0, 0, 0)                              | (terminal region) | 10       |

Table 1: Scenario states on the `medium_tight` grid under the `industry` controller. `Post_Collision` lies inside the terminal region, so its bounded-reachability value must be exactly 1; it is verified as an end-to-end sanity check of the emitted model rather than as a scenario of interest.

**Verification.** For each non-absorbing scenario state of Table 1 we compute the bounded-reachability interval  $[P_{\min}, P_{\max}]$  for reaching a crash within a horizon of  $N = 100$  control periods (10 s at  $\Delta t = 0.1$  s), expressed against the closed-loop encoding as `F<=100 "crash"`. Robust verification is performed with Storm 1.13.0 [?] (official container build, Z3 backend), which recognises the encoding as an interval MDP and applies robust value iteration under the worst-case (for  $P_{\max}$ ) and best-case (for  $P_{\min}$ ) resolution of the intervals. On the production artifact Storm constructs the sparse model in approximately 768 s; the constructed model has 1,977,682 states, one choice per state, confirming the closed-loop structure of (39), and 307,323,579 interval transitions. One value iteration computes the value at every state simultaneously, so all properties of one resolution are evaluated in a single invocation that parses the model once; individual scenario states and whole regions are then read off with property filters over the state labels, for example

```
filter(max, Pmax=? [ F<=100 "crash" ], "nominal").
```

A filtered maximum over a region label is the largest worst-case value over every start state carrying the label. Over `nominal` it is the statement *starting anywhere with no need to brake, the probability of a crash within 10 s is at most the reported value*; the corresponding filtered minimum under the best-case resolution completes the bracket. The same properties over `braking` and `emergency` are statements about the warning region and the emergency region respectively, in the sense fixed in Section 5.1. The `Post_Collision` filter is the sanity check, required to return exactly 1. The maximise batch completed in 10,115.606 s of wall-clock time with a peak memory of 37,515 MB, as reported by the checker’s own resource measurement; the minimise batch completed in 5,923.478 s at a peak of 37,518 MB.

**Results.** Table 2 reports the campaign. Three observations follow. First, the sanity check passes: the `Post_Collision` state, inside the terminal region, returns exactly 1. Second, the `Imminent_Collision` bracket is  $[1, 1]$ : every transition distribution consistent with the certified intervals reaches a crash within the horizon, so by the argument of Section 4.3 the modelled system crashes from that state with probability exactly 1. The result is physically expected: the scenario closes at 30 m/s over a 5 m gap, while the strongest response the plant can realise, the emergency action through the actuator gain of (32), has a stopping distance of roughly 92 m. Third, every other bracket is  $[0, 1]$ , the vacuous interval, and this is the saturation anticipated by the displacement-budget analysis of Section 4.6: the intervals admit a drift of up to one gap cell, 1 m, per period, so over 100 periods the budget spans the entire 100 m gap axis, letting the worst-case resolution reach the crash wall from every reported start state and the best-case resolution avoid it from every reported start state except `Imminent_Collision`. The saturated

brackets are sound and carry information about the abstraction rather than the system: at this resolution, the ten-second horizon exceeds what the intervals can decide. Two experiments follow directly: computing the brackets as functions of the step bound, which locates the largest horizon this resolution decides (Section 5.4), and refining the grid over a reduced window (Section 5.6).

| Start set                           | Kind           | $P_{\min}$ | $P_{\max}$ |
|-------------------------------------|----------------|------------|------------|
| Safe_Cruising                       | scenario state | 0          | 1          |
| Warning_Brake                       | scenario state | 0          | 1          |
| Emergency_Brake                     | scenario state | 0          | 1          |
| Imminent_Collision                  | scenario state | 1          | 1          |
| Post_Collision                      | sanity check   | —          | 1          |
| <b>nominal</b>                      | region         | 0          | 1          |
| <b>braking</b> (warning region)     | region         | 0          | 1          |
| <b>emergency</b> (emergency region) | region         | 0          | 1          |

Table 2: Bounded-reachability results for  $F \leq 100$  "crash" (10s at  $\Delta t = 0.1$ s) on the certified closed-loop encoding. Scenario rows report the bracket at the single labelled state. Region rows report the envelope over the region: the smallest best-case value and the largest worst-case value over every start state carrying the label. Post\_Collision is the sanity check of Table 1, evaluated under the worst-case resolution only. Values are exactly as reported by the checker. Provenance: partition fingerprint 3672963e58eb75b5, code revision 3b9f3e9, containment run identity and equivalence certificate of this section.

#### 5.4 Horizon-resolved verification on the production grid

The saturation of Table 2 is horizon-dependent, so we query the same certified model at a ladder of step bounds, the *horizon rungs*

$$k \in \{5, 10, 15, 20, 25, 30, 40, 50, 75\}; \quad (40)$$

*rung spacing* denotes the difference between successive queried bounds. Each rung is one property  $F \leq k$  "crash" against the certified closed-loop encoding of Section 5.3; one Storm invocation evaluates a batch of rungs after constructing the model once.

Table 3 reports the frontier. Three findings. First,  $P_{\max}$  transitions from 0 to 1 within a single rung for every start set at this spacing, so the decidable horizon is located to one rung. Second,  $P_{\min} = 0$  at all 36 start-set and rung combinations, as the structural analysis of Section 5.3 predicts; the zero lower bounds are horizon-resolved, not an artefact of the ten-second query. Third, the ordering of the crossings follows the distance to the terminal region, consistent with the budget rule of Section 4.6.

| Start set               | last rung with $P_{\max} = 0$ | first rung with $P_{\max} = 1$ |
|-------------------------|-------------------------------|--------------------------------|
| Emergency_Brake         | 5                             | 10                             |
| Warning_Brake           | 15                            | 20                             |
| Safe_Cruising           | 30                            | 40                             |
| <b>nominal</b> (region) | 5                             | 10                             |

Table 3: Horizon-resolved frontier on the production grid (**medium\_tight**), rungs as in (40). Every measured transition is  $0 \rightarrow 1$  within one rung; no intermediate value was observed at this spacing.  $P_{\min} = 0$  at all 36 combinations. Imminent\_Collision is  $[1, 1]$  from the first rung onwards and is omitted. Provenance as in Table 2.

Provenance and cost. The maximise batch terminated early after 21 of 36 properties; the three affected cells (Safe\_Cruising at  $k \in \{40, 50, 75\}$ ) were recovered by a separate invocation,

and every value in Table 3 has a checker log line.  $P_{\max}$  cells above a measured value of 1 are implied by monotonicity of  $P_{\max}$  in  $k$  and were not re-run. Model construction from the direct encoding costs approximately 710s per invocation; per-property checking time grows approximately linearly in  $k$ , from 65s at  $k = 5$  to 787s at  $k = 75$ . The minimise batch (36 properties) completed in 7,926.832s at a peak of 37,516 MB; the recovery invocation completed in 2,229.7s at a peak of 37,514 MB.

The frontier sharpens the reading of Table 2: the production resolution decides Warning\_Brake safety to 1.5s and Safe\_Cruising safety to 3s, and returns no fractional value anywhere. Locating fractional brackets requires finer resolution; the next two subsections report where and how.

### 5.5 Resolution sweep

Four exploratory grids isolate the effect of resolution. All use the kernel, controller, and lead of Section 5.3 over a reduced window,  $g \in [0, 100]$ ,  $v_{\text{ego}} \in [0, 16]$ ,  $v_{\text{lead}} \in [-1, 3]$ ; Imminent\_Collision lies outside this window by design, its bracket being already decided as  $[1, 1]$  on the production grid. Each sweep grid passed the file-against-file equivalence check of Section 5.3 before verification. The sweep grids carry *no* containment certificate: their measurements are methodology evidence for the rules of Section 4.6, and only the two certified configurations ship results.

| Preset | $(r_g, r_{v_{\text{ego}}}, r_{v_{\text{lead}}})$ | Role                                  |
|--------|--------------------------------------------------|---------------------------------------|
| r100   | (1.0, 0.5, 0.1)                                  | production resolution, reduced window |
| r050   | (0.5, 0.5, 0.1)                                  | gap halved                            |
| r025   | (0.25, 0.5, 0.1)                                 | gap quartered                         |
| r050v  | (0.5, 0.25, 0.1)                                 | ego halved (control-visibility probe) |

Table 4: Sweep presets. Window  $g \in [0, 100]$ ,  $v_{\text{ego}} \in [0, 16]$ ,  $v_{\text{lead}} \in [-1, 3]$  throughout; equivalence-checked, not containment-certified.

Three measurements follow.

1. *Crossings move outward with gap resolution.* From r100 to r050: Warning\_Brake  $15 \rightarrow 20$ , Safe\_Cruising  $40 \rightarrow 50$ , nominal  $5 \rightarrow 10$ . From r050 to r025 the crossings are unchanged at 5-step rung spacing. The first fractional certificate of the study appears here:  $P_{\max} = 0.9846419946$  for Safe\_Cruising at  $k = 50$  on r050.
2. *Finer rungs sharpen the frontier.* At 1–2 step rung spacing, Safe\_Cruising transitions between 48 and 52 on r050 and between 55 and 60 on r025; Warning\_Brake transitions between 21 and 23 on r050 and between 23 and 25 on r025, with  $P_{\max} = 0.9999807082$  at  $k = 25$ . The apparent one-rung walls of Table 3 are a property of the rung spacing, and gap refinement pays with diminishing returns.
3. *The ego probe confirms the control-visibility rule.* On r050v, with  $r_{v_{\text{ego}}} = 0.25$  below the emergency threshold of 0.49 m/s, Safe\_Cruising returns the fractional certificate  $P_{\max} = 0.999865419$  at  $k = 52$ , a rung at which r050 ( $r_{v_{\text{ego}}} = 0.5$ , both thresholds failed) has already saturated at 1. Finer ego resolution makes the emergency response visible to the grid and removes part of the worst case available at the coarser ego resolution.

The linear budget (29) matches the measured crossings at five crossings across the two gap-resolution grids; Emergency\_Brake transitions later than predicted, so the heuristic errs on the conservative side. The diagnostic  $K = 6.276171$  of Section 5.3 recurs identically on r050v, at a different worst state, confirming its spatial uniformity and grid-independence for this kernel.

### 5.6 A second certified configuration: the Following window

The production grid decides frontiers but returns no fractional value (Section 5.4), and the sweeps show that fractional certificates appear when the resolution rules of Section 4.6 are satisfied

(Section 5.5). This subsection applies all four rules at once: a certified configuration at resolution 0.1 in every dimension, over a window sized to the cell budget, on a scenario family chosen to produce scenario-discriminating fractional brackets.

**Locating the family: nominal margin search.** A noise-free simulation sweep of the closed loop locates the crash boundary: ego speeds  $\{12, 14, 16\}$  m/s behind a lead held at 10 m/s, starting gaps 4 to 26 m in 0.5 m steps, 60 s per run, 135 runs. Every run settles into steady following; none crashes. The worst minimum gap per ego row is 3.24, 1.66, and 1.12 m, and the steady settle gaps are approximately 5.3, 3.6, and 3.1 m. Two conclusions. The nominal closed loop is collision-free across this envelope, so any certified crash probability from these start states is noise-induced. And start states at small margins above the settle gap are the natural carriers of fractional probabilities: close enough for lead-velocity noise to matter, far enough that a crash is not forced.

**Scenario family.** Five start states at ego 14 m/s behind a lead at 10 m/s, graded by starting gap (Table 5). The closing speed is 4 m/s, below the controller’s 5 m/s branch threshold, so the first decision is distance-based: Following-Critical starts inside the braking band ( $g < 6$ ) and the other four start coasting. Following-Critical also lies inside the warning region of Section 5.1.

| Scenario           | $(g, v_{\text{ego}}, v_{\text{lead}})$ | First action | State id |
|--------------------|----------------------------------------|--------------|----------|
| Following-Critical | (5, 14, 10)                            | brake        | 621200   |
| Following-Tight    | (6, 14, 10)                            | coast        | 743410   |
| Following-Near     | (8, 14, 10)                            | coast        | 987830   |
| Following-Mid      | (10, 14, 10)                           | coast        | 1232250  |
| Following-Wide     | (12, 14, 10)                           | coast        | 1476670  |

Table 5: The Following family on the `extra_fine` grid. All states share ego 14 m/s and lead 10 m/s; the starting gap is the single graded variable. State identifiers are functions of the partition fingerprint below.

**Configuration.** The `extra_fine` preset instantiates the rules of Section 4.6:  $r_{v_{\text{ego}}} = 0.1$  passes both control-visibility thresholds (0.2 and 0.49 m/s),  $\sigma_v/r_{v_{\text{lead}}} \approx 2.0$  is unchanged, and the window is sized to the cell budget.

| Parameter                                                        | Value                                   |
|------------------------------------------------------------------|-----------------------------------------|
| Grid resolution $(r_g, r_{v_{\text{ego}}}, r_{v_{\text{lead}}})$ | (0.1, 0.1, 0.1)                         |
| Bounds $(g, v_{\text{ego}}, v_{\text{lead}})$                    | $[0, 16] \times [4, 16] \times [5, 15]$ |
| Grid shape $(n_g, n_{v_{\text{ego}}}, n_{v_{\text{lead}}})$      | (161, 121, 101)                         |
| Number of cells $M$                                              | 1,967,581                               |
| Partition fingerprint                                            | 8252bc93d8ad810a                        |
| Code revision                                                    | 2ff1c85                                 |
| Kernel, controller, lead, tail policy                            | as in Section 5.3                       |

The window trades coverage for resolution: the static-family scenarios of Table 1 lie outside it by design and continue to report from the production grid. The two configurations report jointly: the production grid decides the static family, and the Following window carries the fractional results below.

**Certificates.** The containment test of Section 4.3 visited all 1,967,581 cells under all three actions and completed with zero violations and a worst recorded excess of 0. The equivalence check of Section 5.3 passed over 1,967,582 states and 311,491,148 transitions, every label included; the canonical hash 915b2f5c23372916f1f48ba83405db2a1824b25d1b69cc511e5a4d13ea44861c was recorded identically for the composed modular model and the direct encoding. Generation completed in 2,596.24 s.

**Results.** Table 6 and Figure 17 report  $P_{\max}$  for  $F \leq k$  "crash" at rungs  $k = 10$  to 24 in steps of 2; coarser rungs at  $k \in \{30, 40, 50, 60\}$  return 1 for all five scenarios.  $P_{\min} = 0$  at all 44 measured scenario-rung combinations, as the structural analysis of Section 5.3 requires. Four findings.

1. *Every scenario carries fractional certificates.* Each column contains at least one strictly fractional  $P_{\max}$ ; the definition of done for this track, fractional and scenario-discriminating brackets, is met.
2. *The ordering follows the starting gap in every cell.* At every rung,  $P_{\max}(\text{Critical}) \geq P_{\max}(\text{Tight}) \geq P_{\max}(\text{Near}) \geq P_{\max}(\text{Mid}) \geq P_{\max}(\text{Wide})$ , with strict inequality wherever the values are not both saturated.
3. *Saturation staggers by one metre of gap.* The first rung with  $P_{\max} = 1$  is 12, 14, 18, 20, 24 across the five scenarios (Figure 18).
4. *Transitions span roughly four rungs.* At 2-step spacing each scenario passes from below  $10^{-10}$  to 1 over four to six rungs, resolving the one-rung walls of Table 3 into curves.

| $k$ | Critical (5 m)         | Tight (6 m)             | Near (8 m)              | Mid (10 m)              | Wide (12 m)             |
|-----|------------------------|-------------------------|-------------------------|-------------------------|-------------------------|
| 10  | $1.859 \times 10^{-3}$ | $1.903 \times 10^{-12}$ | 0                       | 0                       | 0                       |
| 12  | 1                      | 0.4891                  | $2.923 \times 10^{-15}$ | 0                       | 0                       |
| 14  | 1                      | 1                       | $1.504 \times 10^{-4}$  | $6.379 \times 10^{-22}$ | 0                       |
| 16  | 1                      | 1                       | 0.9894                  | $1.835 \times 10^{-5}$  | $4.952 \times 10^{-28}$ |
| 18  | 1                      | 1                       | 1                       | 0.2399                  | $7.816 \times 10^{-6}$  |
| 20  | 1                      | 1                       | 1                       | 1                       | 0.2771                  |
| 22  | 1                      | 1                       | 1                       | 1                       | 0.9998                  |
| 24  | 1                      | 1                       | 1                       | 1                       | 1                       |

Table 6:  $P_{\max}$  for  $F \leq k$  "crash" across the Following family on the certified `extra_fine` encoding. Values rounded to four significant figures; the exact checker output is preserved in the artifact logs.  $P_{\min} = 0$  at every measured combination. Provenance: partition fingerprint 8252bc93d8ad810a, code revision 2ff1c85, certificates of this section.

**Cost.** The fine-rung maximise batch (24 properties) completed in 3,494.9 s of wall-clock time at a peak of 30,878 MB; the coarse batches complete within the resource envelope of Section 5.3. The window trade of Section 4.6 is thus realised at no increase in verification cost over the production grid.

**Simulation corroboration.** A Monte Carlo campaign simulates the modelled system directly: the plant equations of Section 5.1, the controller quantised at cell centres, and the lead-velocity kernel truncated to the grid band and renormalised, matching the certified modelling kernel. A crash is entry into the crash cell slab; departure from the window ends a run as safe, mirroring the sink routing. Each scenario ran 10,000 trajectories to  $k = 24$  under a fixed recorded seed; the simulator's controller logic is asserted against the implementation at startup, and the script is

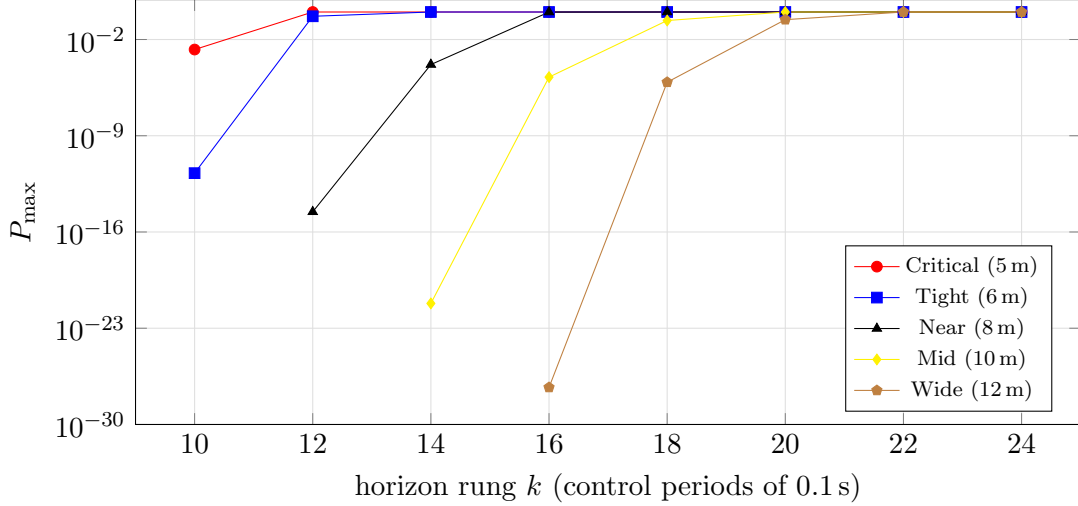


Figure 17: Worst-case crash probability against the horizon rung for the Following family, logarithmic vertical axis. Each point is one checker result on the certified `extra_fine` encoding; rungs at which  $P_{\max}$  is exactly 0 cannot be drawn on a logarithmic axis and are omitted (Table 6 records them). The curves shift right with the starting gap without crossing.

preserved with the artifacts. Every empirical crash fraction lies within its certified bracket. The crash counts are zero at every measured rung except Following\_Near, which recorded 2 crashes in 10,000 by  $k = 18$ : fraction  $2.0 \times 10^{-4}$ , Wilson 95% interval  $[5.5 \times 10^{-5}, 7.3 \times 10^{-4}]$ . Two readings follow. First, containment is corroborated on sampled trajectories, independently of the exhaustive per-cell test. Second, the campaign measures the one-sidedness of the brackets: at rungs where  $P_{\max}$  lies between 0.24 and 1, the empirical fractions are at most  $2.0 \times 10^{-4}$ , with 95% upper confidence limits at most  $7.3 \times 10^{-4}$ , and at Following\_Critical,  $k = 10$ , the upper confidence limit  $3.8 \times 10^{-4}$  falls below  $P_{\max} = 1.86 \times 10^{-3}$  itself. The fractional  $P_{\max}$  values therefore quantify the worst case admissible within the certified intervals, not the likelihood of the modelled system; the distance between the two is the one-sidedness identified in Section 5.3 and taken up in Section 5.7.

## 5.7 Future work

The perfect-perception abstraction is the baseline of the wider programme (Section 1). Track 2 quantifies the perception components into per-input uncertainty tuples; Track 3 replaces the identity perception module of Section 4.4 with an enumeration of perception outputs weighted by those tuples, producing the end-to-end model from which runtime monitors are synthesised. The noise state-independence trip-wire of Algorithm 2 marks exactly the point at which that composition will force the interval computation to range over state-dependent noise. The Following brackets of Section 5.6 are the quantitative baseline against which the perception-aware model’s brackets will be compared.

Within the present track, two items remain. First, the nondeterministic lead: a lead that chooses per step from a set of admissible accelerations [?] requires a second action axis in the discretiser and the composition, over which the checker would quantify adversarially; it is defined in the implementation but not composed. Second, the width structure of the brackets. The zero lower bounds are exact for width-preserving deterministic dynamics (Section 5.3), so every measured  $P_{\min}$  in this paper is 0 and the reported brackets are one-sided: informative upper bounds, trivial lower bounds. Three levers could produce informative lower bounds: abstractions whose transitions target *sets* of states rather than individual cells, which preserve the coherence the per-target intervals discard [?]; game-based abstractions, which separate the abstraction’s nondeterminism from the system’s and tighten both bounds [?]; and actuation noise on the ego

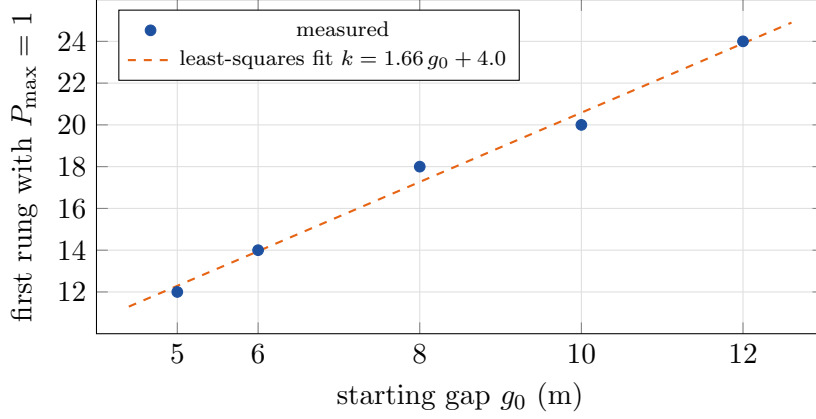


Figure 18: Saturation rung against starting gap for the Following family. The dashed line is a descriptive least-squares fit,  $k \approx 1.66 g_0 + 4.0$ ; its reciprocal slope, 0.60 m of starting gap per period, is the empirical worst-case closure rate of the certified intervals for this family. The fit is descriptive, not a bound.

dynamics, which would make the ego dimension stochastic and remove the straddling indicators that force the zeros, at the price of a second stochastic dimension in the kernel. [UNVERIFIED: the characterisations of [?] and [?] are to be checked against the sources before circulation.]

## 6 Conclusion

We have documented a partition-based IMDP abstraction pipeline for a stochastic hybrid AEBS model under perfect perception. The construction follows the partition-based framework in its overall architecture and adopts its per-transition range bound on the stochastic dimension [?]; it departs from that framework by targeting an interval-valued model, by enumerating exact deterministic images through corner evaluation in the style of mixed-monotone reachability [?], and by certifying soundness through exhaustive per-transition containment under the Markov set-chain preservation result [?, ?] rather than through an additive error bound, which is structurally uninformative here (Sections 3, 4.3). Soundness rests on a single hypothesis, the containment of the modelled kernel’s transition probabilities within the stored intervals, and that hypothesis is discharged by test on the shipped artifacts: zero violations across all cells and actions on both certified configurations.

The verification campaigns quantify what the certified models decide. On the production grid, robust model checking certifies a crash probability of exactly 1 from the imminent-collision scenario, and the horizon-resolved campaign locates the decidable horizon of every other scenario to one rung at five-step spacing, with  $P_{\min} = 0$  throughout as the width analysis predicts. The resolution sweep ties the frontier positions to the design rules of Section 4.6. The second certified configuration then delivers what the production resolution cannot: fractional, scenario-discriminating brackets across the five-state Following family, ordered by starting gap at every measured rung, saturating one metre of gap at a time (Figures 17 and 18). Together the two configurations report a division of labour fixed by the resolution rules: the coarse grid decides the static scenario family over a wide window, and the fine grid carries the fractional results over a narrow one. These brackets are the perfect-perception baseline into which the subsequent tracks of the programme will inject perception-side uncertainty, and against which the end-to-end, monitor-synthesising model will be measured.